

**Исследование и реализация
виртуализации
интеллектуальной станции
мониторинга окружающей
среды на базе STM32**

Научно-исследовательская работа

10 мая 2026 г.

АННОТАЦИЯ

Новый виток информационной революции, представленный Интернетом вещей и встроенными технологиями, кардинально меняет модель функционирования общества, а интеллектуальный мониторинг окружающей среды стал ключевым компонентом при строительстве умных городов. Однако в процессе обучения и разработки встроенных систем разработчики сталкиваются с рядом серьёзных проблем: высокая стоимость оборудования, сложность процесса отладки, высокий порог понимания принципов работы датчиков и неудобства визуализации данных. Традиционный подход к разработке встроенных систем требует от обучающихся приобретения большого количества аппаратных устройств и многократной отладки на реальных печатных платах, что не только увеличивает затраты на обучение, но и ограничивает гибкость и масштабируемость учебного процесса.

Для решения вышеуказанных проблем в настоящей работе предложена и реализована система виртуализации интеллектуальной станции мониторинга окружающей среды на базе STM32. Основные вклады данного исследования включают три аспекта: во-первых, спроектирована и реализована высокодостоверная виртуальная аппаратная платформа STM32F103C8T6, полностью имитирующая ядро МК, систему памяти, GPIO, АЦП, I2C, UART и таймеры, что обеспечивает программную среду для выполнения встроенных программ без реального оборудования; во-вторых, на базе виртуальной платформы разработаны виртуальные драйверы для шести типовых датчиков окружающей среды: DHT11 (температура и влажность), MQ-135 (качество воздуха), BH1750 (освещённость), BMP280 (атмосферное давление) и датчик шума, а также создана унифицированная платформа моделирования датчиков и модель шума, обеспечивающие достоверность и надёжность имитационных данных; в-третьих, построена веб-система визуализации на базе Python Flask, реализующая функции сбора, хранения, оповещения и графического отображения данных датчиков в реальном времени, взаимодействующая с фронтендом через RESTful API и поддерживающая удалённое управление параметрами окружающей среды.

Результаты экспериментов показывают, что эффективность выполнения инструкций виртуальной платформы STM32 имеет высокую степень согласованности с реальным оборудованием, точность моделирования датчиков достигает или приближается к номинальным значениям, указанным в технической документации реальных датчиков, время отклика API веб-системы составляет менее 100 мс, что удовлетворяет требованиям мониторинга в реальном времени. Данное исследование предоставляет для обучения встроенным системам экспериментальную платформу с низким порогом входа и высокой

гибкостью, а также создаёт эффективную среду программной разработки для прототипирования приложений Интернета вещей, обладая положительной ценностью как в теоретическом, так и в инженерном аспектах.

Ключевые слова: STM32; виртуализация; мониторинг окружающей среды; имитационное моделирование датчиков; Интернет вещей; обучение встроенным системам

Содержание

Аннотация	1
1 Введение	4
1.1 Актуальность и значимость исследования	4
1.2 Состояние исследований в области	5
1.2.1 Исследования платформ имитационного моделирования встроенных систем	5
1.2.2 Исследования систем мониторинга окружающей среды	7
1.3 Основное содержание работы	8
1.4 Структура работы	9
2 Общее проектирование системы	9
2.1 Анализ функциональных требований	9
2.1.1 Функциональные требования	9
2.1.2 Нефункциональные требования	11
2.2 Проектирование общей архитектуры	11
2.3 Выбор технологий	13
3 Проектирование виртуальной аппаратной платформы STM32	15
3.1 Проектирование виртуального ядра МК	16
3.1.1 Обзор параметров STM32F103C8T6	16
3.1.2 Моделирование таймера SysTick	16
3.2 Проектирование системы памяти	17
3.2.1 Модель отображения памяти	17
3.2.2 Реализация режима little-endian	18
3.2.3 Отображение регистров	18
3.3 Моделирование периферийных устройств	19
3.3.1 Универсальный ввод-вывод GPIO	19
3.3.2 Аналого-цифровой преобразователь АЦП	20
3.3.3 Контроллер шины I2C	21
3.3.4 Последовательная связь UART	22
3.3.5 Универсальный таймер TIM	24
4 Драйверы датчиков и сбор данных	26
4.1 Принципы виртуализации датчиков	26
4.1.1 Универсальная моделирующая платформа	26
4.1.2 Интерфейс драйвера датчика	27

4.2	Датчик температуры и влажности DHT11	28
4.2.1	Обзор датчика	28
4.2.2	Моделирование протокола однопроводной шины . . .	28
4.3	Датчик качества воздуха MQ-135	30
4.3.1	Принцип работы датчика	30
4.3.2	Обратный расчёт концентрации по напряжению . . .	30
4.3.3	Реализация виртуального драйвера	31
4.4	Датчик освещённости BH1750	31
4.4.1	Обзор датчика	31
4.4.2	Формула расчёта освещённости	32
4.5	Датчик атмосферного давления BMP280	33
4.5.1	Обзор датчика	33
4.5.2	Проверка идентификатора чипа	33
4.5.3	24-разрядный АЦП и коэффициенты калибровки . . .	33
4.6	Датчик шума	34
4.6.1	Принцип работы датчика	34
4.6.2	Линейное отображение напряжения	35
4.7	Проектирование цикла сбора данных	35
5	Проектирование и реализация программного обеспечения	38
5.1	Модуль хранения данных	38
5.1.1	Структура таблиц базы данных	38
5.1.2	Интерфейс Python sqlite3	40
5.1.3	Операции CRUD	40
5.1.4	Статистические запросы	41
5.2	Проектирование системы оповещений	42
5.2.1	Настройка порогов оповещений	42
5.2.2	Логика проверки оповещений	43
5.2.3	Сохранение записей об оповещениях	44
5.3	Веб-система визуализации	45
5.3.1	Проектирование маршрутов Flask	45
5.3.2	Формат ответов RESTful JSON	46
5.3.3	Настройка двойного линейного графика Chart.js . . .	47
5.3.4	Механизм опроса с периодом 1 с	48
5.4	Проектирование панели управления	49
5.4.1	Проектирование механизма межпроцессного взаимодействия	49
5.4.2	Разбор команды set	50
5.4.3	Ограничение диапазона параметров	51

6	Тестирование и верификация системы	52
6.1	Тестовая среда	52
6.2	Функциональное тестирование	53
6.3	Производительное тестирование	55
6.3.1	Время отклика API	55
6.3.2	Стабильность периода сбора данных	55
6.3.3	Использование памяти	56
6.4	Проверка точности датчиков	56
6.4.1	Сравнительный анализ точности	57
6.4.2	Долгосрочное тестирование стабильности	57
6.5	Проверка функции оповещений	58
6.5.1	Сценарий оповещения о высокой температуре	58
6.5.2	Сценарий оповещения о низком давлении	59
6.5.3	Сценарий одновременного возникновения нескольких оповещений	59
6.6	Демонстрация веб-интерфейса	59
7	Заключение и перспективы	61
7.1	Итоги работы	61
7.2	Перспективы дальнейших исследований	62
	Список литературы	65

1 Введение

1.1 Актуальность и значимость исследования

В настоящее время новый виток информационной революции, представленный Интернетом вещей (Internet of Things, IoT), кардинально меняет модель функционирования общества. Реализация таких направлений, как умные города, промышленная автоматизация, точное земледелие и интеллектуальные здания, невозможна без непрерывного мониторинга параметров физической среды и интеллектуального анализа полученных данных. Показатели температуры, влажности, качества воздуха, освещённости, атмосферного давления и уровня шума напрямую влияют не только на здоровье и комфорт человека, но и на энергоэффективность производственных процессов. Поэтому разработка стабильных, экономически доступных и гибких решений для интеллектуального мониторинга окружающей среды обладает значительной ценностью как в социальной, так и в экономической сфере.

Встроенные вычислительные системы составляют техническую основу периферийных узлов Интернета вещей, отвечая за получение, предварительную обработку и передачу данных с датчиков. Линейка микроконтроллеров STM32 компании STMicroelectronics, в частности модель STM32F103C8T6 на базе ядра Cortex-M3, благодаря сбалансированному соотношению производительности и энергопотребления, разнообразию встроенной периферии и зрелости инструментария разработки широко используется в университетском образовании и инженерной практике. Данный чип накопил богатый опыт применения в таких областях, как мониторинг окружающей среды, промышленное управление и потребительская электроника.

Однако как в образовательном процессе, так и при самостоятельном обучении освоение встроенных систем сопряжено с рядом объективных трудностей:

(1) **Высокая стоимость оборудования.** Для комплектации полноценной системы мониторинга окружающей среды необходимо приобретать отладочную плату STM32, модули различных датчиков, программатор, источник питания и соединительные провода; стоимость одного комплекта обычно составляет от нескольких сотен до тысяч юаней. Для университетских лабораторий оснащение десятками полных комплектов экспериментального оборудования требует значительных капитальных вложений; для индивидуальных обучающихся высокие начальные затраты также создают определённый барьер.

(2) **Сложность процесса отладки.** Отладка встроенных систем напрямую зависит от фактического состояния аппаратных соединений: любое ослабление

контакта, неправильная пайка резистора или ошибка в конфигурации вывода может привести к полному отказу системы. Начинающие разработчики при столкновении со сложными аппаратными неисправностями зачастую не в состоянии быстро локализовать причину проблемы, в результате чего значительное время уходит на поиск неисправностей в аппаратуре вместо изучения программных алгоритмов и системной логики.

(3) **Высокий порог понимания датчиков.** Принципы работы, коммуникационные протоколы и методы обработки данных различных датчиков существенно различаются. Например, DHT11 использует однопроводный протокол для передачи 40-битных данных, MQ-135 требует считывания напряжения через АЦП и последующего обратного расчёта концентрации газа по сложной математической модели, а BMP280 задействует коммуникацию по I2C, считывание 24-битного АЦП и алгоритм температурной компенсации. Глубокое понимание механизмов работы этих датчиков требует от обучающихся знаний в области аналоговой и цифровой схемотехники, коммуникационных протоколов и обработки данных.

(4) **Неудобства визуализации данных.** В традиционных средах разработки встроенных систем данные с датчиков обычно выводятся через последовательный порт в текстовом формате, и разработчикам приходится просматривать исходные числовые значения в терминальной программе, что лишено наглядных средств визуализации. Даже при использовании программного обеспечения верхнего уровня часто требуется дополнительная разработка, а функциональные возможности оказываются ограниченными, что затрудняет реализацию таких продвинутых функций, как удалённый веб-доступ, запрос исторических данных и интеллектуальные оповещения.

На основании вышеизложенного разработка системы виртуализации интеллектуальной станции мониторинга окружающей среды на базе STM32, позволяющей обучающимся проходить весь цикл разработки встроенных систем без приобретения физического оборудования, обладает двойной ценностью: с одной стороны, она предоставляет для учебных курсов недорогую экспериментальную среду, с другой — создаёт среду быстрой итерации для ранней проверки концепций.

1.2 Состояние исследований в области

1.2.1 Исследования платформ имитационного моделирования встроенных систем

Исследования в области виртуализации процессоров и имитационного моделирования встроенных систем начались достаточно давно, и как

академическое сообщество, так и промышленность вложили в них значительные усилия. QEMU (Quick Emulator) в качестве открытого универсального эмулятора машин поддерживает множество архитектур наборов команд, включая серию ARM Cortex-M. Его основной механизм — динамическая бинарная трансляция, позволяющая эффективно преобразовывать поток инструкций целевой платформы в поток инструкций хоста, вследствие чего QEMU часто используется при портировании операционных систем и верификации встроенного ПО. Однако акцент эмуляции QEMU делается на ядре ЦПУ и поведении операционной системы, а поддержка моделирования периферийных устройств микроконтроллеров (GPIO, АЦП, ТИМ и др.) остаётся несовершенной; кроме того, процесс конфигурации и кривая обучения не вполне соответствуют требованиям учебного процесса и быстрой разработки прототипов.

Proteus, разработанный британской компанией Labcenter Electronics, представляет собой инструмент автоматизации проектирования электроники (EDA), в который встроены возможности схемотехнического моделирования, разработки печатных плат и совместной отладки микроконтроллеров. Данное ПО поддерживает имитацию микроконтроллеров серии STM32 и интегрирует виртуальные приборы и осциллографы для вспомогательного анализа, широко применяясь в электротехнических дисциплинах высших учебных заведений. Однако Proteus распространяется по коммерческой лицензии с высокой стоимостью, а скорость имитации ограничивается точностью на уровне схемы; при увеличении масштаба системы производительность заметно снижается.

На внутреннем уровне такие университеты, как Цинхуа и Чжэцзян, провели перспективные исследования в области виртуальных сред для экспериментов со встроенными системами. Некоторые учебные заведения создали веб-платформы для удалённых встроенных экспериментов, позволяющие студентам через браузер получать доступ к реальному оборудованию, развёрнутому в вычислительных центрах, и выполнять программирование и онлайн-отладку. Хотя такие решения обеспечивают сетевой доступ к аппаратным ресурсам, на нижнем уровне они по-прежнему привязаны к физическим устройствам и не устраняют корневые затраты на закупку оборудования и ограничения по его количеству.

В последние годы проникновение языка Python в научные исследования и образование продолжает расти, и чисто программные платформы обучения встроенным системам на основе высокоуровневых языков начали привлекать внимание исследователей. Такие решения используют Python для описания поведения ядра МК и периферии, обладая преимуществами короткого

цикла разработки, удобного расширения функциональности и кроссплатформенно, однако по точности временных характеристик и производительности они всё ещё уступают низкоуровневым эмуляторам.

1.2.2 Исследования систем мониторинга окружающей среды

На уровне датчиков и мониторингового оборудования международные производители выпустили несколько поколений коммерческих продуктов. Серия сенсоров температуры и влажности SHT компании Sensirion (США), серия барометрических сенсоров BMP компании Bosch (Германия) и серия газовых сенсоров CCS компании AMS (Австрия) представляют передовой уровень в отрасли. Эти изделия характеризуются высокой точностью измерений, низким энергопотреблением и стандартизированными цифровыми интерфейсами, широко применяясь в профессиональных приборах мониторинга окружающей среды.

Внутренние исследовательские институты также достигли значительного прогресса в данном направлении. Китайская академия наук, Цинхуа и другие научные учреждения провели систематические исследования в области мониторинга атмосферных загрязнителей, контроля качества воды и измерения параметров почвы, создав ряд мониторинговых устройств с самостоятельной интеллектуальной собственностью. На потребительском рынке интеллектуальные устройства мониторинга окружающей среды, выпускаемые компаниями Xiaomi, Huawei и другими производителями, обычно интегрируют многотиповые сенсорные элементы и модули Wi-Fi связи; пользователи могут в реальном времени просматривать показатели окружающей среды через мобильные приложения, что пользуется высоким спросом на рынке.

В области облачных платформ такие зарубежные сервисы, как ThingSpeak и Adafruit IO, а также внутренние платформы OneNET и IoT-платформа Alibaba Cloud предоставляют относительно полную инфраструктуру для удалённого хранения, вычислительного анализа и графического представления мониторинговых данных. Указанные платформы обычно используют протокол MQTT для подключения терминалов и встроенные механизмы правил и уведомлений о событиях.

Следует отметить, что существующие исследовательские результаты и коммерческие продукты в основном ориентированы на реальные сценарии развёртывания и уделяют недостаточное внимание специфическим когнитивным закономерностям и экспериментальным потребностям учебного процесса. Вопрос абстракции и виртуализации процесса разработки систем мониторинга окружающей среды для адаптации к учебным целям курсов по встроенным

системам по-прежнему требует глубокого изучения.

1.3 Основное содержание работы

Настоящая работа посвящена виртуализации интеллектуальной станции мониторинга окружающей среды на базе STM32; исследование ведётся по четырём направлениям:

(1) Проектирование и реализация виртуальной аппаратной платформы STM32. На базе модели STM32F103C8T6 спроектирована полноценная виртуальная аппаратная платформа микроконтроллера, включающая модули имитации ядра Cortex-M3, системы отображения памяти, таймера SysTick, универсального ввода-вывода GPIO, аналого-цифрового преобразователя АЦП, контроллера шины I2C, последовательного порта UART и универсального таймера TIM. Платформа реализована на языке Python, обладая хорошей читаемостью кода, расширяемостью и кроссплатформенностью.

(2) Разработка виртуальных драйверов датчиков окружающей среды. На базе виртуальной аппаратной платформы разработаны виртуальные драйверы для пяти типовых датчиков окружающей среды: DHT11, MQ-135, BH1750, BMP280 и датчик шума. Создана унифицированная платформа моделирования датчиков, включающая определение параметров окружающей среды, модель преобразования физических величин и модель гауссовского шума, что обеспечивает соответствие имитационных данных физическим закономерностям и наличие определённой случайности, приближая их к характеристикам выходных сигналов реальных датчиков.

(3) Построение веб-системы визуализации данных. На базе фреймворка Python Flask разработаны серверные службы; для хранения данных датчиков, записей об оповещениях и системных журналов используется база данных SQLite. Спроектированы интерфейсы RESTful API для вызова фронтендом. Фронтенд реализован на нативных технологиях HTML/CSS/JavaScript с использованием библиотеки Chart.js для построения графиков в реальном времени, отображения статистической информации и функций панели оповещений.

(4) Механизм управления параметрами окружающей среды и взаимодействия. Спроектирован механизм межпроцессного взаимодействия (IPC) на базе совместно используемых файлов: через файл env_control.txt реализуется двусторонняя связь между веб-панелью управления и виртуальным МК. Пользователь может в реальном времени корректировать параметры окружающей среды (температуру, влажность, давление и др.) через веб-интерфейс, а виртуальные датчики на основании обновлённых параметров

формируют соответствующие имитационные данные, реализуя замкнутое взаимодействие.

1.4 Структура работы

Работа состоит из семи глав, организованных следующим образом:

Глава 1 представляет собой введение, в котором изложены актуальность и значимость исследования, состояние исследований в области, основное содержание работы и структура.

Глава 2 посвящена общему проектированию системы: анализ функциональных и нефункциональных требований, проектирование четырёхуровневой общей архитектуры и обоснование выбора ключевых технологий.

Глава 3 раскрывает проектирование виртуальной аппаратной платформы STM32: подробно изложены принципы имитации и методы реализации виртуального ядра МК, системы памяти и различных периферийных устройств.

Глава 4 посвящена драйверам датчиков и сбору данных: представлена универсальная моделирующая платформа виртуализации датчиков, подробно проанализированы принципы имитации и реализация драйверов для каждого датчика.

Глава 5 охватывает проектирование и реализацию программного обеспечения: модуль хранения данных, система оповещений, веб-система визуализации и панель управления.

Глава 6 посвящена тестированию и верификации системы: функциональное, производительное тестирование и проверка точности датчиков с анализом результатов.

Глава 7 содержит заключение и перспективы: подведены итоги основных результатов работы и сформулированы направления дальнейшего совершенствования.

2 Общее проектирование системы

2.1 Анализ функциональных требований

2.1.1 Функциональные требования

Система должна реализовать виртуальную платформу интеллектуальной станции мониторинга окружающей среды; её функциональные цели охватывают шесть аспектов:

(1) **Имитация виртуального МК:** система должна имитировать основные

функции микроконтроллера STM32F103C8T6, включая выполнение инструкций процессора, доступ к памяти, систему тактирования и управление периферией. Встроенный код на языке C (или эквивалентный код на Python), написанный пользователем, должен корректно выполняться на виртуальной платформе без модификации при переносе на реальное оборудование.

(2) **Сбор данных с множества датчиков:** система должна поддерживать сбор данных как минимум с пяти датчиков окружающей среды: температуры, влажности, качества воздуха (в виде концентрации вредных газов), освещённости, атмосферного давления и уровня шума. Каждый датчик должен иметь независимый драйвер и соответствовать соответствующему коммуникационному протоколу.

(3) **Хранение и управление данными:** система должна обеспечивать персистентное хранение собранных данных датчиков в базе данных с поддержкой запросов по диапазону времени, статистического анализа и экспорта данных. Одновременно должны вестись журналы работы системы и журналы событий оповещений для облегчения диагностики неисправностей и аудита.

(4) **Визуализация данных в реальном времени:** система должна предоставлять веб-интерфейс для отображения данных в реальном времени в форме числовых карточек и линейных графиков, наглядно демонстрирующих текущие показания и исторические тренды каждого датчика. Графики должны поддерживать масштабирование временной оси и подсказки при наведении на точки данных.

(5) **Интеллектуальная функция оповещений:** система должна поддерживать установку верхних и нижних пороговых значений для каждого датчика; при выходе показаний датчика за пределы порогового диапазона должно автоматически генерироваться оповещение, а информация о нём записываться в базу данных и отображаться на фронтенд-панели. Состояние оповещения должно обновляться и сбрасываться в реальном времени.

(6) **Управление параметрами окружающей среды:** система должна позволять пользователю в реальном времени корректировать параметры окружающей среды через веб-интерфейс, имитируя поведение датчиков при различных условиях. Команды управления должны вступать в силу своевременно, а текущие значения параметров среды отображаться на интерфейсе.

2.1.2 Нефункциональные требования

Помимо функциональных целей, работа системы должна соответствовать следующим ограничениям:

(1) **Требования к производительности:** эффективность выполнения инструкций виртуального МК должна быть достаточно высокой, а время обработки одного цикла сбора данных не должно превышать 1 с. Время отклика веб-API должно контролироваться в пределах 100 мс, а частота обновления графиков на фронтенде должна быть не ниже 1 Гц.

(2) **Требования к надёжности:** система должна обеспечивать непрерывную стабильную работу в течение не менее 72 ч без сбоев; операции с базой данных должны сопровождаться механизмом транзакций, предотвращающим потерю или несогласованность данных. Драйверы датчиков должны корректно обрабатывать исключительные ситуации, такие как таймауты связи и ошибки контрольных сумм.

(3) **Требования к расширяемости:** архитектура системы должна обладать хорошим модульным проектированием; при добавлении нового типа датчика или периферийного модуля это должно достигаться путём добавления файлов драйверов и конфигурационных файлов без необходимости модификации основного кода. API веб-бэкенда должны соответствовать спецификации RESTful для облегчения расширения функциональности фронтенда.

(4) **Требования к удобству использования:** система должна сопровождаться чёткой документацией и инструкциями; веб-интерфейс должен иметь разумную компоновку и интуитивно понятное управление. API-интерфейсы виртуального оборудования должны иметь лаконичный дизайн, снижающий порог обучения и сложность разработки для пользователей.

(5) **Требования к кроссплатформенности:** система должна работать в основных операционных системах Windows, Linux и macOS без зависимости от специфических аппаратных или программных особенностей. Виртуальная платформа должна быть реализована на языке Python, используя его кроссплатформенные свойства для обеспечения совместимости.

2.2 Проектирование общей архитектуры

На основании вышеизложенных требований система делится снизу вверх на четыре уровня: уровень аппаратной абстракции, уровень драйверов устройств, уровень бизнес-логики и уровень приложений и представления. Границы функций каждого уровня приведены в таблице 1.

Таблица 1: Четырёхуровневая архитектура системы

Уровень	Название	Описание функций
Уровень 4	Уровень приложений и представления	Отвечает за рендеринг интерфейса фронтенда, построение графиков, пользовательское взаимодействие и отображение данных; реализован на нативном HTML/CSS/JavaScript
Уровень 3	Уровень бизнес-логики	Отвечает за серверные слои Flask, проектирование RESTful API, управление хранением данных, обработку логики оповещений, разбор управляющих команд
Уровень 2	Уровень драйверов устройств	Отвечает за разработку драйверов различных датчиков и реализацию цикла сбора данных; выполнение преобразования физических величин в цифровые
Уровень 1	Уровень аппаратной абстракции	Отвечает за имитацию виртуального МК, системы пако контроллеров шин и различных периферийных устройств; предоставляет интерфейс для уровня регистров оборудования

Уровень аппаратной абстракции (Hardware Abstraction Layer, HAL) составляет фундамент всей платформы; его поведение на уровне регистров согласовано с поведением реального чипа STM32. Уровень драйверов устройств вызывает интерфейсы HAL для инициализации датчиков, считывания данных и разбора протоколов. Уровень бизнес-логики выполняет обработку, персистентное хранение и анализ исходных данных и предоставляет возможности верхнему уровню в форме стандартизированного API. Уровень приложений и представления сосредоточен на оптимизации качества отображения интерфейса и плавности взаимодействия.

Взаимодействие между уровнями осуществляется посредством нормализованных интерфейсных контрактов; модули верхних уровней запрещено напрямую манипулировать внутренним состоянием нижних уровней в обход интерфейсов.

Такой подход эффективно контролирует степень связанности модулей. При необходимости введения нового типа датчика требуется лишь добавить соответствующий файл драйвера на уровне драйверов устройств и зарегистрировать правила разбора на уровне бизнес-логики; при этом уровень аппаратной абстракции и уровень приложений и представления остаются неизменными.

2.3 Выбор технологий

При выборе инструментов и фреймворков в системе придерживались принципа “простота, эффективность, лёгкая расширяемость”; основные технологии сведены в таблицу [2](#).

Таблица 2: Выбор технологий системы

Категория	Технология	Обоснование выбора
Язык разработки	Python 3.11	Лаконичный синтаксис, высокая эффективность разработки, богатая стандартная и сторонние библиотеки, хорошая кроссплатформенность
Веб-фреймворк	Flask 3.0	Лёгкий, гибкий, простой в освоении, активное сообщество, богатые расширения
База данных	SQLite 3	Нулевая конфигурация, хранение в одном файле, не требует отдельного серверного процесса, подходит для приложений среднего и малого масштаба
Фронтенд	Нативный HTML/CSS/JS	Отсутствие зависимостей сборки, высокая скорость загрузки, хорошая совместимость, простота развёртывания
Библиотека графиков	Chart.js 4.x	Открытый исходный код, дружелюбный API, поддержка адаптивного дизайна и анимации
Контроль версий	Git	Распределённое управление версиями, гибкое управление ветками, высокая эффективность совместной работы

Выбор Python 3.11 в качестве базового языка разработки обусловлен тремя соображениями: во-первых, лаконичный синтаксис Python и выдающаяся читаемость исходного кода облегчают построчное объяснение в учебном процессе; во-вторых, стандартная библиотека охватывает типовые задачи сетевого взаимодействия, доступа к базам данных, файловых операций

и параллельного программирования, что снижает необходимость во внешних зависимостях; в-третьих, режим интерпретации исключает время ожидания компиляции, позволяя немедленно проверять изменения и сокращая цикл отладки.

Flask — лёгкий веб-фреймворк на базе Python, философия которого подчёркивает минимализм и гибкость. По сравнению с полнофункциональными фреймворками, такими как Django, Flask не навязывает заранее определённую структуру каталогов проекта и привязку компонентов, позволяя разработчику самостоятельно выбирать базу данных, шаблонизатор и стратегию аутентификации в соответствии с фактическими потребностями. Для веб-приложения масштаба данной системы функциональность Flask находится в разумных пределах, избегая как чрезмерного проектирования, так и недостатка возможностей.

SQLite представляет собой встраиваемую реляционную базу данных; все данные концентрируются в одном файле на диске и не требуют обслуживания отдельным серверным процессом. Такое свойство “готовности к работе сразу после установки” существенно упрощает процесс развёртывания, особенно в сценариях индивидуальной разработки, учебных экспериментов и небольших приложений. SQLite поддерживает стандартный синтаксис SQL и транзакции ACID, что вполне достаточно для удовлетворения требований данной системы в области персистентности и анализа данных.

В качестве фронтенд-технологий были выбраны нативные HTML5, CSS3 и JavaScript без использования компонентных фреймворков, таких как React или Vue. Основной мотивацией данного решения являлся контроль общей сложности, исключение накладных расходов на конфигурацию инструментария сборки и сохранение исходного кода фронтенда в непосредственно читаемом виде. Chart.js в качестве открытой библиотеки компонентов для построения графиков предоставляет разнообразные типы диаграмм и конфигурационные параметры; её движок рендеринга на базе Canvas демонстрирует хорошую производительность, удовлетворяя требованиям к визуализации данных в реальном времени.

3 Проектирование виртуальной аппаратной платформы STM32

Виртуальная аппаратная платформа STM32 составляет нижнюю опору всей системы. Идея её построения заключается в воспроизведении поведения микроконтроллера STM32F103C8T6 на уровне регистров в чисто программной

среде, чтобы драйверы датчиков и код прикладного уровня могли выполняться так же, как на реальном чипе. Ниже приведено конкретное описание имитации ядра МК, механизма отображения памяти и моделирования основных периферийных устройств.

3.1 Проектирование виртуального ядра МК

3.1.1 Обзор параметров STM32F103C8T6

STM32F103C8T6 производства STMicroelectronics оснащён ядром ARM Cortex-M3 и относится к моделям средней плотности линейки STM32F1. Данный чип обеспечивает хороший баланс между производительностью и стоимостью, вследствие чего широко применяется в университетском образовании и инженерной практике. Ключевые технические параметры сведены в таблицу 3.

Таблица 3: Основные технические параметры STM32F103C8T6

Параметр	Спецификация
Архитектура ядра	ARM Cortex-M3
Максимальная частота	72 МГц
Память Flash	64 КБ
SRAM	20 КБ
Выходы GPIO	37 (корпус LQFP48)
АЦП	2 12-разрядных АЦП, 16 каналов
Таймеры	3 универсальных + 1 продвинутый + 2 базовых
Интерфейсы связи	2 I2C, 3 USART, 2 SPI, 1 CAN
Рабочее напряжение	2.0 В – 3.6 В
Рабочая температура	–40°C – +85°C
Тип корпуса	LQFP48

Поскольку данная платформа реализована на языке Python и не выполняет послетактовую имитацию набора команд ARM или тактовых циклов, основной акцент делается на достоверное воспроизведение поведения регистров периферийных устройств. Временная шкала использует 1 мс в качестве минимального шага; задержки и измерение времени выполняются с помощью модуля `time` стандартной библиотеки Python.

3.1.2 Моделирование таймера SysTick

Ядро Cortex-M3 интегрирует SysTick — 24-разрядный счётчик обратного отсчёта, обеспечивающий периодические импульсы для операционной

системы и приложений. В виртуальной платформе, реализованной на Python, имитация SysTick сосредоточена на трёх возможностях: декремент счётчика, установка значения перезагрузки и генерация прерываний. В силу особенностей интерпретируемого языка обеспечить микросекундную точность временных характеристик затруднительно, поэтому применяется приближённая схема на основе относительного времени.

Функция `delay_ms` выполняет задержку на уровне миллисекунд. При входе в функцию фиксируется текущее системное время, после чего выполняется циклическая проверка прошедшего времени; как только оно достигает заданного количества миллисекунд, функция возвращает управление. Такая стратегия активного ожидания загружает процессор, но для учебных демонстраций и функциональной верификации этого достаточно.

```
1 import time
2
3 def delay_ms(ms):
4     """Функция задержки на уровне миллисекунд
5     :
6     :
7     : ms: время задержки в миллисекундах
8     """
9     start = time.time()
10    while (time.time() - start) * 1000 < ms:
11        pass
```

Listing 1: Реализация виртуальной функции `delay_ms`

Код 1 демонстрирует реализацию `delay_ms` с активным ожиданием. В ходе учебного процесса можно организовать обсуждение достоинств и недостатков данной схемы: простота реализации и отсутствие внешних зависимостей являются её преимуществами; невозможность выполнения других задач процессором в период задержки — очевидным недостатком. При необходимости улучшения можно попробовать переписать реализацию на базе `threading` или `asyncio` в неблокирующем варианте.

3.2 Проектирование системы памяти

3.2.1 Модель отображения памяти

STM32F103C8T6 объединяет Flash, SRAM и регистры периферийных устройств в единое 32-битное адресное пространство. Виртуальная система

памяти воспроизводит три ключевых адресных региона:

(1) **Память Flash**: используется для хранения программного кода и константных данных; адресный диапазон начинается с 0x08000000, размер 64 КБ. На виртуальной платформе область Flash в основном применяется для хранения имитируемого микропрограммного кода и конфигурационных параметров.

(2) **SRAM**: используется для хранения переменных времени выполнения, стека и кучи; адресный диапазон начинается с 0x20000000, размер 20 КБ. Область SRAM виртуальной платформы реализована в виде массива байтов с поддержкой чтения и записи по байтам, полусловам и словам.

(3) **Регистры периферийных устройств**: включают конфигурационные и данных регистры периферийных устройств GPIO, АЦП, UART, TIM, I2C и др.; адресный диапазон начинается с 0x40000000. Каждое периферийное устройство занимает непрерывный адресный диапазон, а регистры располагаются с фиксированными смещениями.

Класс MemoryRegion является ядром виртуальной системы памяти, управляя потоком байтов чтения и записи в непрерывном адресном диапазоне. STM32 использует порядок байтов little-endian: младший байт многобайтных данных размещается по младшему адресу, а старший — по старшему; класс MemoryRegion строго соблюдает это правило при операциях чтения и записи.

3.2.2 Реализация режима little-endian

Режим Little-Endian означает, что старший байт данных записывается по старшему адресу, а младший — по младшему. Например, для значения 0x12345678 последовательность байтов в памяти будет 0x78, 0x56, 0x34, 0x12 по мере возрастания адреса.

Модуль struct языка Python предоставляет средства управления порядком байтов: символ формата '<' обозначает little-endian, а '>' — big-endian. Класс MemoryRegion с помощью функций struct.pack и struct.unpack выполняет сериализацию и десериализацию многобайтных данных, обеспечивая согласованность обработки порядка байтов виртуальной платформы с реальным микроконтроллером STM32.

3.2.3 Отображение регистров

RegisterMap представляет собой дополнительный уровень абстракции над MemoryRegion, ориентированный на управление регистрами периферийных устройств. Каждый экземпляр периферийного устройства содержит объект RegisterMap, что позволяет разработчику обращаться к регистрам по

имени, а не по голому адресу, повышая читаемость кода.

RegisterMap предоставляет четыре основные возможности:

(1) **Регистрация регистров:** при инициализации определяются все регистры данного периферийного устройства, включая их названия, смещения адресов, значения по умолчанию и права доступа (только чтение / только запись / чтение-запись).

(2) **Доступ к битовым полям:** поддерживаются операции чтения и записи отдельных битов или битовых полей в регистре. Например, с помощью `set_bit('CR1', 0)` можно установить бит 0 регистра CR1 в 1 без предварительного чтения всего регистра и ручного вычисления.

(3) **Защита записи:** для регистров только для чтения (например, регистр состояния SR) RegisterMap отклоняет прямые операции записи, предотвращая случайное изменение значений регистров, которые не должны модифицироваться.

(4) **Обратные вызовы при изменении:** позволяет регистрировать функции обратного вызова для конкретных регистров; при изменении значения регистра обратный вызов автоматически активируется. Данный механизм важен для имитации поведенческих реакций периферийных устройств: например, при записи в регистр ODR GPIO необходимо немедленно обновить состояние выходов соответствующих выводов.

3.3 Моделирование периферийных устройств

3.3.1 Универсальный ввод-вывод GPIO

GPIO отвечает за ввод и вывод цифровых сигналов, являясь первым рубежом взаимодействия микроконтроллера с внешними схемами. Каждая группа портов GPIO микроконтроллера STM32F103C8T6 содержит 16 выводов (Pin 0 – Pin 15), которые могут независимо конфигурироваться в режимы плавающего входа, входа с подтяжкой к питанию, входа с подтяжкой к земле, аналогового входа либо режимы двухтактного или открытого стока выхода.

Виртуальный GPIO построен на следующих механизмах:

(1) **Массив состояний выводов:** каждый порт GPIO поддерживает массив длины 16, фиксирующий текущий логический уровень каждого вывода (0 или 1). В режиме входа состояние вывода определяется внешним устройством или подтягивающим/стягивающим резистором; в режиме выхода состояние вывода определяется значением регистра ODR.

(2) **Регистр входных данных IDR:** IDR (Input Data Register) используется для чтения текущего логического уровня входов порта GPIO. На виртуальной

платформе значение IDR получается путём кодирования массива состояний выводов. Конкретно, бит i регистра IDR равен текущему логическому уровню вывода номер i .

Формула расчёта IDR:

$$\text{IDR} = \sum_{i=0}^{15} P_i \times 2^i \quad (1)$$

где P_i обозначает логический уровень вывода номер i , $P_i \in \{0, 1\}$.

(3) **Регистр выходных данных ODR**: ODR (Output Data Register) используется для установки логического уровня выходов порта GPIO. При записи нового значения в ODR виртуальный модуль GPIO побитно разбирает его, обновляет состояние выходов соответствующих выводов и активирует зарегистрированные функции обратного вызова.

(4) **Механизм обратных вызовов**: для поддержки реакции внешних устройств на изменения состояния GPIO реализован механизм регистрации обратных вызовов. Пользователь может зарегистрировать функции обратного вызова для событий нарастающего фронта, спадающего фронта или изменения уровня на конкретном выводе. При возникновении соответствующего изменения состояния вывода все зарегистрированные функции обратного вызова выполняются в порядке регистрации. Данный механизм весьма полезен при имитации прерываемого выхода датчиков и управления индикацией светодиодов.

3.3.2 Аналого-цифровой преобразователь АЦП

АЦП отображает непрерывный аналоговый сигнал напряжения в дискретную цифровую величину, выполняя роль переводчика между аналоговым входным каскадом и цифровым backend. Микроконтроллер STM32F103C8T6 интегрирует два 12-разрядных АЦП последовательного приближения (SAR), поддерживающих 16 внешних каналов и 2 внутренних канала (встроенный температурный сенсор и опорное напряжение).

Ключевые свойства виртуального АЦП:

(1) **12-разрядное разрешение**: результат преобразования АЦП представляет собой 12-разрядное целое число без знака в диапазоне от 0 до 4095. Соответствующая формула квантования:

$$D = \left\lfloor \frac{V_{in}}{V_{ref}} \times 4096 \right\rfloor \quad (2)$$

где D — цифровой выход, V_{in} — входное аналоговое напряжение, V_{ref} — опорное напряжение (обычно 3.3 В).

(2) **Преобразование напряжение-цифра**: когда код верхнего уровня считывает данные с определённого канала АЦП, виртуальный АЦП рассчитывает соответствующую цифровую величину по формуле 2 на основании текущего значения аналогового напряжения, приложенного к данному каналу. Значение аналогового напряжения предоставляется подключённым виртуальным датчиком или внешним источником сигнала.

(3) **Модель шума**: для более реалистичного моделирования характеристик АЦП к идеальному результату преобразования добавляется гауссовский белый шум. Амплитуда шума регулируется конфигурационными параметрами; по умолчанию составляет ± 2 младших значащих разрядов (LSB). Формула преобразования с шумом:

$$D_{actual} = D_{ideal} + N(0, \sigma^2) \quad (3)$$

где $N(0, \sigma^2)$ обозначает нормально распределённую случайную величину с нулевым средним и дисперсией σ^2 ; значение σ по умолчанию приблизительно равно 2.

(4) **Выбор и включение каналов**: виртуальный АЦП поддерживает многоканальную конфигурацию; порядок и количество каналов преобразования определяются регистром последовательности SQR (Regular Sequence Register). Перед началом преобразования необходимо включить АЦП (установить бит ADON регистра CR2 в 1), а затем запустить преобразование (установить бит SWSTART регистра CR2 в 1).

3.3.3 Контроллер шины I2C

Шина I2C выполняет много-master много-slave обмен данными по двум линиям SDA и SCL и широко применяется в датчиках, EEPROM и других устройствах. Микроконтроллер STM32F103C8T6 оснащён двумя интерфейсами I2C, совместимыми со стандартным режимом (100 кГц) и быстрым режимом (400 кГц).

Виртуальная шина I2C акцентирует моделирование следующих аспектов поведения:

(1) **Подключение устройств**: виртуальная шина I2C поддерживает список подключённых устройств; каждое устройство однозначно идентифицируется 7-битным адресом slave. Когда код верхнего уровня отправляет через I2C стартовый сигнал и байт адреса, контроллер шины ищет соответствующий экземпляр виртуального устройства по адресу и устанавливает соединение.

Интерфейс подключения устройств определён следующим образом:

```
1 def attach_device(self, address, device):
```



```
2      """Подключает виртуальное устройство к шине
3          I2C Параметры
4      :
5          address: битный 7- адрес slave
6          device: экземпляр виртуального устройства
7                  , должен реализовывать методы
8                      read/write
9      """
10     self.devices[address] = device
```

Listing 2: Интерфейс подключения виртуальных устройств к шине I2C

(2) **Операции чтения и записи:** виртуальная шина I2C поддерживает побайтовые и буферизованные операции чтения и записи. При операции записи master-устройство отправляет адрес slave и бит записи (0), после чего передаёт адрес целевого регистра и байты данных; при операции чтения master-устройство сначала отправляет адрес slave и бит записи для указания адреса регистра, затем повторно отправляет стартовый сигнал, адрес slave и бит чтения (1), после чего slave-устройство возвращает запрошенные байты данных.

(3) **Обработка ошибок:** виртуальная шина I2C реализует базовый механизм обнаружения ошибок, включающий: NACK по адресу — возврат ошибки при отсутствии совпадения отправленного адреса slave в списке устройств; занятость шины — обнаружение конфликта при одновременной попытке нескольких master-устройств захватить шину; проверка данных — опциональная CRC-проверка передаваемых данных.

3.3.4 Последовательная связь UART

UART в качестве асинхронного последовательного интерфейса часто применяется для обмена данными между МК и ПК, Bluetooth-модулями, GPS-приёмниками и другими устройствами. Микроконтроллер STM32F103C8T6 располагает тремя независимыми USART, поддерживающими полный дуплекс и множество настроек скорости передачи.

Виртуальный UART в реализации акцентирует следующие аспекты:

(1) **Настройка скорости передачи:** скорость передачи в бодах определяет количество битов, передаваемых в секунду, и является ключевым параметром связи UART. Виртуальный UART фиксирует текущее значение скорости передачи; хотя в силу программной природы имитации истинное побитовое управление временными характеристиками по скорости передачи невозможно,

значение скорости влияет на некоторые поведения, связанные с временными характеристиками (например, настройки таймаута). Типичные значения скорости передачи включают 9600, 115200, 921600 и др.

(2) **Имитация перенаправления printf**: во встроенной разработке функция стандартного вывода printf обычно перенаправляется на UART для вывода отладочной информации через последовательный порт. Виртуальный UART предоставляет эквивалентный интерфейс отправки строк: передаваемая строка буферизуется во внутреннем буфере и активирует функцию обратного вызова приёмника.

(3) **Механизм обратных вызовов**: виртуальный UART поддерживает регистрацию функции обратного вызова при приёме данных; при поступлении данных функция вызывается автоматически. Данный механизм имитирует прерывание приёма реального UART, позволяя коду верхнего уровня обрабатывать данные порта в событийно-ориентированном режиме без необходимости циклического опроса приёмного буфера.

```
1 class VirtualUART:
2     def __init__(self):
3         self.baudrate = 115200
4         self.rx_callback = None
5         self.tx_buffer = []
6
7     def set_rx_callback(self, callback):
8         """Регистрирует функцию обратного вызова при приёме"""
9         self.rx_callback = callback
10
11     def send(self, data):
12         """Отправляет данные строка ( или байты )"""
13         if isinstance(data, str):
14             data = data.encode('utf-8')
15         self.tx_buffer.extend(data)
16         if self.rx_callback:
17             self.rx_callback(data)
18
19     def printf(self, fmt, *args):
20         """Форматированный вывод, имитация printf"""
21         msg = fmt % args
22         self.send(msg)
```

Listing 3: Регистрация обратного вызова и передача виртуального UART

Код 3 демонстрирует реализацию класса VirtualUART; интерфейс send совместим с типами str и bytes, а printf воспроизводит семантику форматированного вывода языка C.

3.3.5 Универсальный таймер TIM

TIM в семействе периферийных устройств STM32 обладает наибольшим многообразием функций: он может использоваться в качестве таймера, счётчика, генератора ШИМ-сигналов или захвата внешних входов. В данном проекте используются только его базовые возможности таймера, обеспечивающие периодический источник запуска для дискретизации датчиков.

VirtualTIM с помощью фонового потока-демона из модуля threading Python выполняет фоновый отсчёт времени. Демон-поток автоматически завершается при выходе основного потока без необходимости ручной очистки, что идеально соответствует требованиям периодических фоновых задач.

Логика работы виртуального TIM включает три шага:

(1) **Инкремент счётчика:** после запуска TIM фоновый поток с фиксированным интервалом (определяемым предделителем и значением автоперезагрузки) инкрементирует внутренний счётчик. Частота инкремента счётчика рассчитывается по формуле:

$$f_{count} = \frac{f_{clk}}{(PSC + 1) \times (ARR + 1)} \quad (4)$$

где f_{clk} — частота тактирования таймера (обычно 72 МГц), PSC — значение предделителя, ARR — значение автоперезагрузки.

(2) **Периодический обратный вызов:** когда счётчик достигает значения автоперезагрузки, он сбрасывается в ноль и генерируется событие обновления (Update Event). Если зарегистрирована функция периодического обратного вызова, она вызывается в этот момент. В системе мониторинга окружающей среды функция периодического обратного вызова обычно выполняет задачи сбора данных с датчиков, обеспечивая автоматическую дискретизацию с фиксированной частотой.

(3) **Запуск и остановка:** TIM управляется битом CEN регистра CR1: установка CEN в 1 запускает таймер, а сброс останавливает его. Виртуальный TIM при запуске создаёт и запускает фоновый поток-демон; при остановке устанавливает флаг события для уведомления потока о необходимости завершения.

```
1 import threading
2 import time
```

```
3
4 class VirtualTIM:
5     def __init__(self):
6         self.prescaler = 7199      # PSC
7         self.auto_reload = 9999    # ARR
8         self.counter = 0
9         self.running = False
10        self.period_callback = None
11        self._stop_event = threading.Event()
12
13    def set_period_callback(self, callback):
14        """Устанавливает функцию периодического обратного вызова"""
15        self.period_callback = callback
16
17    def _timer_thread(self):
18        """Фоновый поток таймера"""
19        period_ms = (self.prescaler + 1) * (self.auto_reload + 1) /
20                    72000
21        while not self._stop_event.is_set():
22            time.sleep(period_ms / 1000.0)
23            self.counter = 0
24            if self.period_callback:
25                self.period_callback()
26
27    def start(self):
28        """Запускает таймер"""
29        self.running = True
30        self._stop_event.clear()
31        t = threading.Thread(target=self._timer_thread, daemon=True)
32        t.start()
33
34    def stop(self):
35        """Останавливает таймер"""
36        self.running = False
37        self._stop_event.set()
```

Listing 4: Реализация фонового потока-демона виртуального TIM

Код 4 демонстрирует механизм периодического запуска VirtualTIM с помощью фонового потока-демона. Функция `period_callback` выполняется при каждом переполнении периода; изменение значений `prescaler` и

auto_reload позволяет перенастроить период таймера для соответствия различным скоростям дискретизации.

4 Драйверы датчиков и сбор данных

Датчики эквивалентны органам чувств мониторинговой системы, преобразуя различные параметры физической среды в электрические сигналы для обработки микроконтроллером. В чисто программной имитационной среде модель датчика выступает в роли связующего звена между виртуальным оборудованием и прикладным уровнем; степень её реалистичности напрямую влияет на достоверность всей системы. Ниже сначала рассматриваются общие принципы моделирования, а затем подробно анализируются принципы имитации и реализация драйверов для конкретных датчиков.

4.1 Принципы виртуализации датчиков

4.1.1 Универсальная моделирующая платформа

В системе создана унифицированная моделирующая платформа для имитации датчиков, в рамках которой все типы датчиков сводятся к трём этапам: ввод параметров окружающей среды, модель преобразования физических величин и цифровой выход.

Данная платформа включает три ключевых компонента:

(1) **Определение параметров окружающей среды:** каждый датчик ассоциирован с набором переменных параметров окружающей среды, представляющих истинные значения физических величин, измеряемых датчиком. Например, температурный датчик ассоциирован с параметром temperature, а датчик освещённости — с параметром illuminance. Значения параметров окружающей среды могут корректироваться в реальном времени через панель управления, имитируя различные условия.

(2) **Модель преобразования физических величин:** описывает, как параметры окружающей среды преобразуются в выходные физические величины датчика (обычно напряжение, сопротивление или цифровое значение). Модели преобразования различных датчиков могут представлять собой линейную зависимость, степенную зависимость или кусочно-заданную функцию. Модель должна строиться на основании технической документации датчика (Datasheet) и физических принципов, обеспечивая обоснованность имитационных данных.

(3) **Модель шума:** выходной сигнал реального датчика неизбежно

содержит случайный шум, источниками которого являются тепловой шум, дробовой шум и электромагнитные помехи. Для повышения реалистичности имитации к идеальному выходному сигналу датчика добавляется гауссовский белый шум. Математическое выражение модели шума:

$$Y = X + N(0, \sigma^2) \quad (5)$$

где Y — фактическое выходное значение датчика, X — идеальное выходное значение (рассчитанное по модели преобразования физических величин), $N(0, \sigma^2)$ — нормально распределённый случайный шум с нулевым средним и дисперсией σ^2 .

Значение интенсивности шума σ определяется классом точности датчика и типовым сценарием применения. Для высокоточных датчиков (например, BMP280) σ принимает малое значение; для низкоточных (например, MQ-135) — большее. В специальных сценариях можно также добавлять выбросы (Outlier) для имитации отказа датчика или резких изменений среды.

4.1.2 Интерфейс драйвера датчика

Все виртуальные драйверы датчиков совместно используют единый интерфейсный контракт, позволяющий коду верхнего уровня вызывать их без учёта особенностей реализации. Интерфейс определён следующим образом:

- `init()`: инициализация датчика, конфигурация параметров связи и рабочего режима.
- `read()`: считывание текущего измеренного значения датчика; возвращает исходные данные или откалиброванную физическую величину.
- `calibrate()`: выполнение калибровки, обновление калибровочных коэффициентов (применимо).
- `set_environment(env_dict)`: обновление параметров окружающей среды, ассоциированных с датчиком.

Благодаря единообразию интерфейса при добавлении нового типа датчика достаточно написать соответствующий класс драйвера и реализовать четыре метода; при этом цикл сбора данных и веб-уровень отображения остаются без изменений.

4.2 Датчик температуры и влажности DHT11

4.2.1 Обзор датчика

DHT11 интегрирует измерение температуры и влажности в одном чипе и взаимодействует с хостом по однопроводному протоколу. Его отличительные особенности — низкая стоимость, компактные размеры и простота освоения, вследствие чего он чрезвычайно популярен в начальных проектах Интернета вещей. Основные технические характеристики приведены в таблице 4.

Таблица 4: Основные технические параметры DHT11

Параметр	Спецификация
Диапазон измерения (влажность)	20% – 90% RH
Точность измерения (влажность)	±5% RH
Диапазон измерения (температура)	0°C – 50°C
Точность измерения (температура)	±2°C
Разрешение	1% RH / 1°C
Период дискретизации	≥ 1 с
Напряжение питания	3.3 В – 5.5 В
Протокол связи	Однопроводной последовательный

4.2.2 Моделирование протокола однопроводной шины

Однопроводный обмен данными с DHT11 можно разложить на три фазы: инициализация хостом, ответ датчика и побитовая передача данных.

(1) **Инициализация хостом:** МК устанавливает линию данных в низкий уровень на время не менее 18 мс, после чего освобождает линию и ожидает ответа DHT11. Данное действие предназначено для пробуждения датчика из режима ожидания и сигнализирует о предстоящей передаче данных.

(2) **Ответ датчика:** восприняв импульс инициализации, DHT11 сначала выдаёт низкий уровень на 80 мкс, затем высокий уровень на 80 мкс, сообщая хосту о своей готовности. Хост фиксирует данную последовательность импульсов для подтверждения присутствия датчика на линии.

(3) **Передача кадра данных:** DHT11 передаёт показания температуры и влажности в 40-битном кадре. Каждый бит начинается с низкого уровня длительностью 50 мкс; длительность последующего высокого уровня определяет логическое значение: 26–28 мкс соответствуют “0”, около 70 мкс — “1”. 40 бит располагаются в следующем порядке: 8 бит целой

части влажности, 8 бит дробной части влажности, 8 бит целой части температуры, 8 бит дробной части температуры и 8 бит контрольной суммы.

Драйвер VirtualDHT11 полностью воспроизводит описанные временные характеристики протокола; ключевая реализация представлена ниже:

```
1 class VirtualDHT11:
2     def __init__(self, gpio, pin):
3         self.gpio = gpio
4         self.pin = pin
5         self.humidity = 50.0      # влажность по умолчанию    50%
6         self.temperature = 25.0   # температура по умолчанию  25C
7
8     def set_environment(self, env):
9         self.humidity = env.get('humidity', 50.0)
10        self.temperature = env.get('temperature', 25.0)
11
12    def read(self):
13        # имитация сигнала запуска
14        self.gpio.set_mode(self.pin, 'OUTPUT')
15        self.gpio.write(self.pin, 0)
16        delay_ms(20)
17        self.gpio.write(self.pin, 1)
18        delay_us(30)
19        self.gpio.set_mode(self.pin, 'INPUT')
20
21        # имитация сигнала ответа и передачи данных
22        # возвращает влажность (, температура)
23        h_int = int(self.humidity)
24        h_dec = int((self.humidity - h_int) * 10)
25        t_int = int(self.temperature)
26        t_dec = int((self.temperature - t_int) * 10)
27        checksum = (h_int + h_dec + t_int + t_dec) & 0xFF
28
29        data = [h_int, h_dec, t_int, t_dec, checksum]
30        return self.humidity + random.gauss(0, 1.5), \
31               self.temperature + random.gauss(0, 0.8)
```

Listing 5: Сигнал запуска DHT11 и считывание данных

В коде 5 драйвер VirtualDHT11 воспроизводит временные характеристики однопроводной шины путём переключения уровней GPIO. На этапе формирования

данных параметры окружающей среды упаковываются в 40-битный кадр с контрольной суммой; затем по формуле 5 вводится гауссовский шум: стандартное отклонение канала влажности установлено на уровне 1.5% RH, канала температуры — 0.8°C, что соответствует классу точности реального DHT11.

4.3 Датчик качества воздуха MQ-135

4.3.1 Принцип работы датчика

MQ-135 демонстрирует высокую чувствительность к аммиаку, сернистым соединениям, бензольным парам и дыму. Чувствительный слой выполнен в виде полупроводниковой плёнки SnO_2 ; молекулы целевого газа на её поверхности участвуют в реакциях окисления и восстановления, вызывая изменение объёмного сопротивления, которое отражает концентрацию газа.

Между сопротивлением чувствительного элемента R_S датчика MQ-135 и концентрацией газа C существует степенная зависимость:

$$\frac{R_S}{R_0} = A \cdot C^{-B} \quad (6)$$

где R_0 — базовое сопротивление датчика в чистом воздухе, A и B — эмпирические константы, зависящие от типа газа, C — концентрация газа (в единицах ppm). Для типичных сценариев мониторинга качества воздуха обычно принимают $A \approx 116.6$, $B \approx 2.77$.

4.3.2 Обратный расчёт концентрации по напряжению

Внутри модуля MQ-135 делитель напряжения преобразует изменение сопротивления в сигнал напряжения. Выходное напряжение связано с сопротивлением чувствительного элемента соотношением:

$$V_{out} = V_{cc} \cdot \frac{R_L}{R_S + R_L} \quad (7)$$

где V_{cc} — напряжение питания (обычно 5 В), R_L — нагрузочное сопротивление (регулируемый потенциометр).

Отсюда можно выразить сопротивление чувствительного элемента:

$$R_S = R_L \cdot \left(\frac{V_{cc}}{V_{out}} - 1 \right) \quad (8)$$

Подставляя данное выражение в формулу 6, получаем аналитическое

выражение для концентрации газа:

$$C = \left(\frac{A \cdot R_0}{R_S} \right)^{\frac{1}{B}} = \left(\frac{A \cdot R_0 \cdot V_{out}}{R_L \cdot (V_{cc} - V_{out})} \right)^{\frac{1}{B}} \quad (9)$$

4.3.3 Реализация виртуального драйвера

Драйвер VirtualMQ135 в качестве входа использует параметр окружающей среды `air_quality` (в единицах ppm) и по формулам 6 и 9 выполняет обратный расчёт напряжения АЦП. Кроме того, драйвер воспроизводит три типичных поведения реального устройства:

(1) **Имитация прогрева:** датчик MQ-135 после включения питания требует определённого времени прогрева для достижения стабильного рабочего состояния; типичное время прогрева составляет 20 с. Виртуальный драйвер фиксирует время запуска после инициализации и в течение периода прогрева возвращает недействительные данные или данные с пониженной точностью.

(2) **Нелинейная характеристика:** в соответствии со степенной зависимостью между концентрацией газа и сопротивлением существует нелинейная связь. В области низких концентраций чувствительность датчика выше; в области высоких концентраций чувствительность постепенно снижается. Виртуальный драйвер полностью имитирует данную нелинейную характеристику.

(3) **Компенсация температуры и влажности:** выходной сигнал MQ-135 зависит от температуры и влажности окружающей среды. Виртуальный драйвер вводит упрощённые коэффициенты компенсации температуры и влажности, корректирующие значение концентрации в зависимости от текущих температуры и влажности, что повышает достоверность имитационных данных.

4.4 Датчик освещённости BH1750

4.4.1 Обзор датчика

BH1750 по шине I2C напрямую выдаёт цифровое значение освещённости в единицах люкс (lux), исключая необходимость во внешнем АЦП и громоздкой калибровке. Широкий диапазон измерений, высокая точность и простота подключения обеспечивают его массовое развёртывание в системах автоматического освещения, регулировки подсветки и мониторинга окружающей среды.

Режим измерения BH1750 определяется байтом команды; часто используемые команды сведены в таблицу 5.

Таблица 5: Часто используемые команды I2C BH1750

Код команды	Название	Описание функции
0x01	Power On	Команда включения питания, пробуждение датчика
0x00	Power Down	Команда отключения питания, переход в режим низкого энергопотребления
0x10	Continuous H-Res Mode	Непрерывный режим высокого разрешения (разрешение 1 люкс, время измерения 120 мс)
0x11	Continuous H-Res2 Mode	Непрерывный режим высокого разрешения 2 (разрешение 0.5 люкс, время измерения 120 мс)
0x13	Continuous L-Res Mode	Непрерывный режим низкого разрешения (разрешение 4 люкс, время измерения 16 мс)
0x20	One Time H-Res Mode	Однократный режим высокого разрешения

4.4.2 Формула расчёта освещённости

В режиме H-Res BH1750 выдаёт 16-разрядное исходное значение (raw value), которое затем умножается на коэффициент чувствительности 1.2 люкс/отсчёт для получения фактической освещённости:

$$E_v = \text{raw} \times 1.2 \quad (10)$$

где E_v — освещённость (в люксах), raw — 16-разрядные исходные данные, возвращённые датчиком.

При значении raw, равном 500, соответствующая освещённость составляет $500 \times 1.2 = 600$ люкс.

VirtualBH1750 после приёма команды по I2C на основании текущего параметра окружающей среды illuminance выполняет обратный расчёт исходного значения по формуле 10:

$$\text{raw} = \left\lfloor \frac{E_v}{1.2} \right\rfloor + N(0, \sigma^2) \quad (11)$$

где σ принимает значение 3 отсчёта, что соответствует шуму измерения

около 3.6 люкс и согласуется со спецификациями технической документации BH1750.

4.5 Датчик атмосферного давления BMP280

4.5.1 Обзор датчика

BMP280 производства Bosch изготавливается по технологии MEMS и сочетает малые габариты, низкое энергопотребление и высокую точность. Он широко применяется в метеорологических станциях, высотомерах, системах indoor-навигации и стабилизации высоты БПЛА. Основные технические параметры приведены в таблице 6.

Таблица 6: Основные технические параметры BMP280

Параметр	Спецификация
Диапазон измерения давления	300 гПа - 1100 гПа
Точность измерения давления	± 0.12 гПа (тип.)
Диапазон измерения температуры	-40°C - $+85^{\circ}\text{C}$
Точность измерения температуры	$\pm 0.5^{\circ}\text{C}$ (при 25°C)
Абсолютная точность	± 1 гПа
Интерфейс связи	I2C или SPI
Идентификатор чипа	0x58

4.5.2 Проверка идентификатора чипа

BMP280 содержит регистр идентификатора чипа, доступный только для чтения, по адресу 0xD0; значение после сброса составляет 0x58. После подачи питания хост обычно сначала считывает данный регистр для подтверждения присутствия устройства на шине. VirtualBMP280 встроил данное поведение на этапе инициализации: обращение на чтение по адресу 0xD0 всегда возвращает 0x58.

4.5.3 24-разрядный АЦП и коэффициенты калибровки

BMP280 интегрирует 24-разрядный АЦП, квантующий аналоговые выходы датчиков давления и температуры. Эффективная разрядность каналов давления и температуры составляет 20 бит; данные упаковываются в 3-байтовые регистры. Высокое разрешение позволяет фиксировать колебания давления на уровне миллибар.

В реальном BMP280 калибровочные коэффициенты записываются в энергонезависимую память NVM (адреса 0x88–0xA1) при заводской

калибровке. Перед считыванием показаний хост должен сначала извлечь эти коэффициенты, а затем на стороне МК выполнить алгоритм температурной компенсации, включающий члены второго и третьего порядка, для восстановления физических величин из исходных значений АЦП.

Виртуальная платформа выполнила разумное упрощение процесса компенсации, аппроксимируя зависимость между давлением, температурой и исходными значениями АЦП линейной функцией:

$$\text{raw_P} = \left\lfloor \frac{P}{1013.25} \times 415148 \right\rfloor + N(0, \sigma_P^2) \quad (12)$$

$$\text{raw_T} = \lfloor (T + 40) \times 100 \rfloor + N(0, \sigma_T^2) \quad (13)$$

где P — значение атмосферного давления окружающей среды (в гПа), T — значение температуры окружающей среды (в °C), 415148 — типичное исходное значение АЦП, соответствующее стандартному атмосферному давлению 1013.25 гПа, σ_P и σ_T — стандартные отклонения шума.

Вывод формулы 12 основан на эмпирическом факте: в режиме передискретизации $\times 16$ типичное исходное значение АЦП, соответствующее стандартному давлению 1013.25 гПа, составляет приблизительно 415148. Установленная таким образом пропорциональная зависимость между давлением и исходным значением хотя и опускает сложную температурную компенсацию и нелинейные поправки, присущие реальному чипу, но обеспечивает достаточную точность для учебных демонстраций и типовой верификации.

4.6 Датчик шума

4.6.1 Принцип работы датчика

Звуковой датчик (типовые представители — модуль KY-038 или модуль на базе компаратора LM393) измеряет интенсивность окружающего шума; результат представляется в децибелах (дБ). Такие модули обычно состоят из высокочувствительного электретного микрофона и схемы обработки сигнала, преобразующей звуковое давление в аналоговое напряжение.

Децибел — логарифмическая единица отношения. Уровень звукового давления SPL (Sound Pressure Level) определяется по формуле:

$$L_p = 20 \cdot \log_{10} \left(\frac{p}{p_0} \right) \quad (14)$$

где L_p — уровень звукового давления (в дБ), p — фактическое звуковое давление (в Па), p_0 — опорное звуковое давление (обычно 20×10^{-6} Па,

т.е. порог слышимости человеческого уха на частоте 1 кГц).

4.6.2 Линеиное отображение напряжения

После усиления и выпрямления модуль датчика выдаёт аналоговое напряжение, пропорциональное звуковому давлению. В виртуальной модели дополнительно предполагается линейная зависимость между напряжением и уровнем звукового давления:

$$V_{out} = V_{offset} + k \cdot L_p \quad (15)$$

где V_{offset} — смещение напряжения (соответствует выходу при 0 дБ; на практике обычно небольшое положительное значение), k — коэффициент чувствительности (в В/дБ).

Драйвер VirtualNoise считывает параметр окружающей среды `noise_level` (в дБ), по формуле 15 рассчитывает выходное напряжение, которое затем квантуется АЦП в цифровое значение. Для приближения к реальным условиям к конечному результату добавляется гауссовский шум со стандартным отклонением 1.5 дБ, что соответствует типичной точности бытовых шумомеров.

VirtualNoise дополнительно воспроизводит следующие три физические характеристики:

(1) **Направленность**: реальный микрофон обладает определённой направленностью, проявляя разную чувствительность к звуку, приходящему с разных направлений. Виртуальный датчик вводит упрощённый коэффициент направленности для имитации данной физической характеристики.

(2) **Частотная характеристика**: человеческое ухо по-разному воспринимает звуки различной частоты; А-взвешивание (A-weighting) является наиболее распространённым способом взвешивания уровня звукового давления. Виртуальный датчик по умолчанию выдаёт А-взвешенный уровень звукового давления, более близкий к субъективному восприятию человека.

(3) **Фиксация пикового значения**: некоторые шумомеры поддерживают функцию пикового обнаружения, фиксирующую максимальный уровень звукового давления за определённый интервал времени. Виртуальный датчик реализует опциональный режим фиксации пикового значения, записывающий и возвращающий максимальное значение в дБ в заданном временном окне.

4.7 Проектирование цикла сбора данных

Цикл сбора данных управляет основным потоком виртуальной мониторинговой станции: с фиксированным периодом активирует каждый датчик для

дискретизации, после чего передаёт результаты в цепочку хранения и отображения. Одиночный цикл разбивается на шесть этапов:

1. **Считывание параметров окружающей среды:** из совместно используемого файла `env_control.txt` считываются текущие настройки параметров окружающей среды, включая температуру, влажность, давление, освещённость, качество воздуха и уровень шума.
2. **Обновление среды датчиков:** считанные параметры окружающей среды передаются экземплярам виртуальных датчиков, позволяя им формировать имитационные данные на основании актуальных настроек.
3. **Выполнение считывания датчиков:** последовательно вызываются методы `read()` датчиков для получения текущих измеренных значений. Обычный порядок считывания: DHT11 (температура/влажность) → BMP280 (давление/температура) → BH1750 (освещённость) → MQ-135 (качество воздуха) → датчик шума.
4. **Форматирование данных:** исходные показания датчиков преобразуются в стандартные единицы (люкс, гПа, ppm, дБ и др.) и организуются в словарь или формат JSON в соответствии с предопределённой структурой данных.
5. **Хранение данных:** отформатированная запись данных вставляется в таблицу `sensor_data` базы данных SQLite; одновременно обновляется кэш последних данных в оперативной памяти.
6. **Проверка оповещений:** выполняется обход конфигурации оповещений для каждого датчика с проверкой, не выходит ли текущее показание за пределы порогового диапазона. При обнаружении превышения записывается журнал оповещения и обновляется его статус.

Для предотвращения чрезмерного удлинения отдельного цикла дискретизации, которое могло бы вызвать дрожание периода, в системе введена стратегия компенсации периода. Пусть целевой период равен T_{sample} (в секундах), а фактическое время выполнения текущего цикла — $t_{elapsed}$; тогда время ожидания рассчитывается по формуле:

$$t_{sleep} = \max(0, T_{sample} - t_{elapsed}) \quad (16)$$

Формула 16 гарантирует, что фактический период не будет короче заданного. Если одиночная дискретизация уже превысила лимит времени,

ожидание пропускается и выполняется немедленный переход к следующему циклу с приоритетом обеспечения актуальности данных.

```
1 import time
2
3 SAMPLE_PERIOD = 2.0 # период дискретизации 2 секунды
4
5 def data_collection_loop(sensors, db, alarm_config):
6     """Главный цикл сбора данных"""
7     while True:
8         t_start = time.time()
9
10        # Шаг 1: считывание параметров окружающей среды
11        env = read_environment_control()
12
13        # Шаг 2: обновление среды датчиков
14        for sensor in sensors:
15            sensor.set_environment(env)
16
17        # Шаг 3: выполнение считывания датчиков
18        readings = {}
19        for sensor in sensors:
20            readings.update(sensor.read())
21
22        # Шаг 4: форматирование данных
23        record = format_sensor_data(readings)
24
25        # Шаг 5: хранение данных
26        db.insert_sensor_data(record)
27
28        # Шаг 6: проверка оповещений
29        check_alarms(record, alarm_config, db)
30
31        # компенсация периода
32        t_elapsed = time.time() - t_start
33        t_sleep = max(0, SAMPLE_PERIOD - t_elapsed)
34        time.sleep(t_sleep)
```

Listing 6: Ядро реализации цикла сбора данных

Код 6 демонстрирует полную реализацию функции `data_collection_loop`. Константа `SAMPLE_PERIOD` определяет период дискретизации; логика

компенсации периода подавляет временной дрейф. Словарь readings переносит промежуточные результаты между этапами; отформатированная запись record в итоге сохраняется в базе данных и участвует в проверке оповещений.

5 Проектирование и реализация программного обеспечения

Виртуальная аппаратная платформа STM32 и драйверы датчиков составляют нижнюю базу выполнения; поверх них четыре ключевых модуля — модуль хранения данных, система оповещений, веб-система визуализации и панель управления — работают совместно, выполняя функции персистентности данных, обнаружения аномалий, отображения результатов и удалённого управления, формируя полноценное программное приложение интеллектуальной станции мониторинга.

5.1 Модуль хранения данных

Мониторинговая система должна сначала записывать собранные показания датчиков, состояние работы и события оповещений в персистентный носитель, чтобы обеспечить возможность последующего просмотра истории, статистического анализа и аудита журналов. SQLite в качестве движка базы данных данной системы не требует отдельного серверного процесса; вся база данных хранится в одном файле, что обеспечивает удобство развёртывания и миграции.

5.1.1 Структура таблиц базы данных

База данных содержит три ключевые таблицы: sensor_data (таблица данных датчиков), alarm_log (таблица записей об оповещениях) и system_log (таблица системных журналов).

(1) **Таблица sensor_data:** предназначена для хранения данных, собранных в реальном времени с датчиков; это таблица с наибольшим объёмом данных. Её структура представлена в таблице 7; таблица содержит 10 полей.

Таблица 7: Структура таблицы sensor_data

Имя поля	Тип данных	Ограничения		Описание
id	INTEGER	PRIMARY	KEY	Автоинкрементный первичный ключ
timestamp	DATETIME	NOT NULL	DEFAULT CURRENT_TIMESTAMP	Метка времени сбора
temperature	REAL			Значение температуры, °C
humidity	REAL			Значение влажности, % RH
pressure	REAL			Значение давления, гПа
illuminance	REAL			Освещённость, люкс
air_quality	REAL			Индекс качества воздуха, ppm
noise_level	REAL			Уровень шума, дБ
cpu_temp	REAL			Внутренняя температура МК, °C
data_source	TEXT	DEFAULT	'simulation'	Идентификатор источника данных

Поле id реализует стратегию автоинкрементного первичного ключа, назначая каждой записи уникальный идентификатор; поле timestamp автоматически заполняется текущим временем при вставке, обеспечивая временную базу для последующего поиска по периодам; поля temperature – noise_level поочерёдно отображают шесть физических величин окружающей среды; поле cpu_temp сохраняет имитационные показания встроенного температурного сенсора виртуального МК; поле data_source по умолчанию помечено как 'simulation', что оставляет расширительный интерфейс для будущего подключения реального оборудования.

(2) **Таблица alarm_log:** отслеживает полный жизненный цикл оповещения от возникновения до снятия. Значения полей следующие: id — первичный ключ; timestamp — фиксация момента первого срабатывания оповещения; sensor_type — тип датчика, вызвавшего оповещение; alarm_type — различие HIGH (верхний порог) и LOW (нижний порог); threshold — установленное граничное значение при срабатывании; actual_value — показание датчика в момент срабатывания; status — текущее состояние ACTIVE (активно)

или RESOLVED (снято); `resolved_at` — метка времени снятия оповещения.

(3) **Таблица `system_log`**: фиксирует ключевые события и отладочный вывод в период работы системы. Поля: `id` — первичный ключ; `timestamp` — момент создания журнала; `level` — четырёхуровневая градация серьёзности: DEBUG, INFO, WARNING, ERROR; `module` — функциональный модуль, сгенерировавший запись; `message` — конкретный текстовый контент; `metadata` — дополнительная структурированная информация в формате JSON для последующего автоматизированного разбора.

5.1.2 Интерфейс Python sqlite3

Стандартная библиотека Python `sqlite3` полностью инкапсулирует С-интерфейс SQLite, позволяя разработчику напрямую выполнять SQL-запросы, управлять границами транзакций и читать результирующие наборы, избавляя от необходимости установки дополнительных драйверов баз данных.

Каждое обращение к базе данных оборачивается контекстным менеджером; соединение автоматически освобождается по завершении операции, исключая риск утечки дескрипторов. Режим `journal_mode` явно установлен в WAL (Write-Ahead Logging), благодаря чему операции чтения и записи могут выполняться параллельно; при одновременном доступе к одной базе данных со стороны потока сбора данных и веб-службы время ожидания блокировок существенно сокращается.

5.1.3 Операции CRUD

Модуль хранения данных предоставляет внешний набор интерфейсов CRUD, охватывающий создание, чтение, обновление и удаление:

(1) **Create (создание)**: `insert_sensor_data` принимает показания датчиков в виде словаря и записывает их в таблицу `sensor_data`. SQL-запрос использует параметризованную привязку; внешний код не может изменить семантику SQL путём конструирования вредоносного ввода, что блокирует путь инъекционных атак.

(2) **Read (чтение)**: модуль предоставляет многомерные возможности поиска: `get_latest_reading()` возвращает последнюю точку дискретизации; `get_readings_by_range(start,end)` извлекает все записи за заданный временной интервал; `get_readings_by_sensor(sensor_type,limit)` выполняет поиск по типу датчика, возвращая последние N исторических записей.

(3) **Update (обновление)**: `update_alarm_status` переводит оповещения в состоянии ACTIVE в состояние RESOLVED и устанавливает поле `resolved_at` равным текущему системному времени, завершая цикл оповещения.

(4) **Delete (удаление)**: `delete_old_readings(before_date)` удаляет устаревшие записи по границе даты; операция может запускаться как автоматически по расписанию, так и вручную администратором, предотвращая непрерывный рост объёма исторических данных.

5.1.4 Статистические запросы

Помимо базовых операций CRUD, модуль встроил возможности агрегированного анализа, позволяющие выполнять статистическую обработку данных дискретизации за определённый период. Типичные агрегированные измерения:

(1) **Запрос среднего значения (AVG)**: вычисляет арифметическое среднее показаний датчика за выбранный временной интервал, характеризуя общий уровень параметра окружающей среды за данный период.

(2) **Запрос минимального значения (MIN)**: извлекает минимальное значение дискретизации за период, помогая выявить аномально низкие значения или экстремальные режимы работы.

(3) **Запрос максимального значения (MAX)**: извлекает максимальное значение дискретизации за период, помогая выявить аномально высокие значения или экстремальные режимы работы.

(4) **Запрос стандартного отклонения (STDEV)**: измеряет степень разброса точек дискретизации относительно среднего; чем больше значение, тем интенсивнее колебания параметров окружающей среды.

Шаблон SQL для статистических запросов:

```
1 SELECT
2     AVG(temperature) as avg_temp,
3     MIN(temperature) as min_temp,
4     MAX(temperature) as max_temp,
5     AVG(humidity) as avg_humidity,
6     MIN(pressure) as min_pressure,
7     MAX(pressure) as max_pressure
8 FROM sensor_data
9 WHERE timestamp BETWEEN ? AND ?;
```

Listing 7: Шаблон SQL статистических запросов

Указанный SQL-шаблон с помощью агрегатных функций одновременно суммирует статистические показатели для нескольких датчиков. Границы временного интервала в предложении WHERE передаются через параметризованные placeholders, что сохраняет гибкость запросов и исключает инъекционные риски, связанные со строковой конкатенацией.

5.2 Проектирование системы оповещений

Система оповещений непрерывно сканирует текущие показания датчиков и при обнаружении выхода за границы генерирует уведомление, напоминая операторам о необходимости своевременного реагирования на аномалии окружающей среды.

5.2.1 Настройка порогов оповещений

Словарь ALARM_CONFIG для каждого датчика поддерживает независимые верхнюю и нижнюю границы: high_threshold определяет верхний порог, low_threshold — нижний. При превышении показанием верхнего порога генерируется оповещение типа HIGH; при снижении ниже нижнего порога — оповещение типа LOW; данные категории не пересекаются.

Пример конфигурации оповещений по умолчанию:

```
1 ALARM_CONFIG = {
2     'temperature': {
3         'high_threshold': 35.0,    #
4             порогоповещениявысокойтемпературе      35C
5         'low_threshold': 10.0,    #
6             порогоповещениянизкойтемпературе      10C
7         'unit': 'C',
8         'name': 'температура'
9     },
10    'humidity': {
11        'high_threshold': 80.0,    #
12            порогоповещениявысокойвлажности      80%
13        'low_threshold': 20.0,    #
14            порогоповещениянизкойвлажности      20%
15        'unit': '% RH',
16        'name': 'влажность'
17    },
18    'pressure': {
19        'high_threshold': 1050.0, #
20            порогоповещениявысокомдавлении      1050 гПа
21        'low_threshold': 980.0,    #
22            порогоповещениянизкомдавлении      980 гПа
23        'unit': 'hPa',
24        'name': 'давление'
25    },
26    'air_quality': {
27        'high_threshold': 500.0,    #
28            порогоповещениявысокогокачества      500
29        'low_threshold': 100.0,    #
30            порогоповещениянизкогокачества      100
31        'unit': 'AQI',
32        'name': 'качество воздуха'
33    }
34 }
```

```
                порогоповещенияокачествевоздуха      500 ppm
22         'low_threshold': None,      # нижнийпороготсутствует
23         'unit': 'ppm',
24         'name': 'качество воздуха'
25     },
26     # ... прочиеконфигурациидатчиков
27 }
```

Listing 8: Пример конфигурации порогов оповещений

Как показано в коде 8, конфигурация каждого датчика включает числовые значения порогов, единицы измерения, название на русском языке и др. Если для датчика `low_threshold` установлен в `None`, то мониторинг нижнего порога не выполняется; отслеживается только верхний порог.

5.2.2 Логика проверки оповещений

Функция `check_all` управляет всем процессом проверки оповещений: последовательно извлекаются конфигурации датчиков из `ALARM_CONFIG`, текущие показания сравниваются с установленными пороговыми интервалами, а результаты сравнения инициируют или снимают оповещения. Конкретные шаги:

(1) **Поэлементная проверка:** извлекаются записи конфигурации для каждого датчика из `ALARM_CONFIG`; текущее показание сравнивается с установленным для данного датчика пороговым интервалом.

(2) **Проверка верхнего порога:** когда показание превышает `high_threshold`, система проверяет, существует ли для данного датчика активное оповещение типа `HIGH` в состоянии `ACTIVE`: если существует — обновляется `actual_value`, если нет — вставляется новая запись `ACTIVE`.

(3) **Проверка нижнего порога:** при снижении показания ниже `low_threshold` выполняется симметричная процедура генерации оповещения типа `LOW`.

(4) **Снятие оповещения:** после возврата показания в пороговый интервал система массово переводит все `ACTIVE`-оповещения данного датчика в состояние `RESOLVED` и записывает метку времени `resolved_at`.

(5) **Персистентность:** как при создании нового оповещения, так и при изменении состояния немедленно вызывается интерфейс базы данных для записи на диск, обеспечивая полную прослеживаемость жизненного цикла оповещений.

```
1 def check_all(readings, config, db):
2     """Проверяет состояние оповещений для всех датчиков. Параметры
3
```

```
4      :
5      readings: словарь текущих показаний датчиков
6      config: словарь конфигурации оповещений
7      db: экземпляр операций с базой данных
8      """
9      for sensor, cfg in config.items():
10         value = readings.get(sensor)
11         if value is None:
12             continue
13
14         # проверка верхнего порога
15         high = cfg.get('high_threshold')
16         if high is not None and value > high:
17             trigger_alarm(db, sensor, 'HIGH', high, value)
18
19         # проверка нижнего порога
20         low = cfg.get('low_threshold')
21         if low is not None and value < low:
22             trigger_alarm(db, sensor, 'LOW', low, value)
23
24         # проверка снятия
25         if (high is None or value <= high) and \
26            (low is None or value >= low):
27             resolve_alarm(db, sensor)
```

Listing 9: Ядро логики проверки оповещений

Как показано в коде 9, функция `check_all` реализует полный цикл проверки оповещений. Вспомогательные функции `trigger_alarm` и `resolve_alarm` отвечают за создание нового оповещения и снятие существующего соответственно.

5.2.3 Сохранение записей об оповещениях

Запись об оповещении сохраняет полный контекст момента срабатывания, включая тип датчика, граничное значение порога и фактическое показание, обеспечивая исходные данные для последующего анализа и аудита. Модель данных описана таблицей 8.

Таблица 8: Структура таблицы alarm_log

Имя поля	Тип данных	Ограничения	Описание
id	INTEGER	PRIMARY KEY AUTOINCREMENT	Автоинкрементный первичный ключ
timestamp	DATETIME	NOT NULL	Время срабатывания оповещения
sensor_type	TEXT	NOT NULL	Тип датчика
alarm_type	TEXT	NOT NULL	Тип оповещения: HIGH/LOW
threshold	REAL	NOT NULL	Пороговое значение срабатывания
actual_value	REAL	NOT NULL	Фактическое значение при срабатывании
status	TEXT	DEFAULT 'ACTIVE'	Состояние: ACTIVE/RESOLVED
resolved_at	DATETIME		Время снятия

5.3 Веб-система визуализации

Веб-уровень визуализации служит мостом между пользователем и виртуальной мониторинговой станцией: данные с нижележащих датчиков визуализируются посредством графиков и приборных панелей, а пользователь через панель управления корректирует параметры окружающей среды и просматривает уведомления об аномалиях на панели оповещений.

5.3.1 Проектирование маршрутов Flask

Flask выполняет функции серверной веб-службы, предоставляя набор RESTful API для потребления фронтендом. Метод HTTP, путь и назначение каждой конечной точки сведены в таблицу 9.

Таблица 9: Проектирование маршрутов RESTful API Flask

Метод	Путь	Функция	Описание
GET	/	Главная страница	Возвращает веб-страницу визуализации
GET	/api/sensors/latest	Последние данные	Возвращает последние показания всех датчиков
GET	/api/sensors/history	История	Возвращает исторические данные за указанный период
GET	/api/alarms/active	Активные оповещения	Возвращает список неснятых оповещений
GET	/api/alarms/history	История оповещений	Возвращает все записи оповещений (с пагинацией)
POST	/api/control	Управление средой	Принимает команды установки параметров окружающей среды
GET	/api/stats	Статистика	Возвращает статистику за указанный период

5.3.2 Формат ответов RESTful JSON

Каждый API возвращает тело JSON, содержащее три обязательных ключа верхнего уровня: status, data, message. Ключ status принимает значения 'ok' или 'error', отражая результат обработки запроса; data содержит фактическую бизнес-нагрузку; message заполняется читаемым описанием ошибки только в исключительных ситуациях.

Пример успешного ответа:

```

1 {
2   "status": "ok",
3   "data": {
4     "temperature": 26.5,
5     "humidity": 58.2,
6     "pressure": 1013.8,
7     "illuminance": 320.0,
8     "air_quality": 120.5,
9     "noise_level": 45.3,
10    "timestamp": "2024-01-15T09:30:00"
11  },
12  "message": null

```

13 }

Listing 10: Пример успешного ответа API

5.3.3 Настройка двойного линейного графика Chart.js

На фронтенде для рендеринга кривых трендов в реальном времени используется библиотека Chart.js. Для одновременного отображения на одном полотне изменений двух связанных датчиков система применяет двойной линейный график с двумя осями Y (Dual Line Chart). Ниже представлен код инициализации Chart.js для совместного отображения температуры и влажности:

```
1 const ctx = document.getElementById('tempHumChart').getContext('2d');
2 const tempHumChart = new Chart(ctx, {
3   type: 'line',
4   data: {
5     labels: [],      // массив временных меток
6     datasets: [
7       {
8         label: 'Температура (C)',
9         data: [],
10        borderColor: 'rgb(255, 99, 132)',
11        backgroundColor: 'rgba(255, 99, 132, 0.1)',
12        yAxisID: 'y-temp',
13        tension: 0.3
14      },
15      {
16        label: 'Влажность (% RH)',
17        data: [],
18        borderColor: 'rgb(54, 162, 235)',
19        backgroundColor: 'rgba(54, 162, 235, 0.1)',
20        yAxisID: 'y-humidity',
21        tension: 0.3
22      }
23    ]
24  },
25  options: {
26    responsive: true,
27    interaction: { mode: 'index', intersect: false },
28    scales: {
29      'y-temp': {
```

```
30         type: 'linear',
31         display: true,
32         position: 'left',
33         title: { display: true, text: 'Температура (C)' }
34     },
35     'y-humidity': {
36         type: 'linear',
37         display: true,
38         position: 'right',
39         title: { display: true, text: 'Влажность (% RH)' }
40     }
41 }
42 }
43 });
```

Listing 11: Настройка двойного линейного графика Chart.js

Двойной линейный график с помощью `yAxisID` отображает кривую температуры на левой оси Y, а кривую влажности — на правой, устраняя проблему визуального сжатия, вызванную различностью единиц измерения. Параметр `tension`, установленный в 0.3, сохраняет характерные точки перегиба данных и одновременно делает линии более плавными.

5.3.4 Механизм опроса с периодом 1 с

Фронтенд посредством `setInterval` с периодом 1000 мс выполняет опрос бэкенда, активно запрашивая последние показания датчиков и состояние оповещений, обеспечивая квазиреальное обновление интерфейса.

```
1 // опроска каждые 1000 мс (1 с)
2 setInterval(() => {
3     fetch('/api/sensors/latest')
4         .then(r => r.json())
5         .then(data => {
6             if (data.status === 'ok') {
7                 updateSensorCards(data.data);
8                 updateCharts(data.data);
9             }
10        })
11        .catch(err => console.error('Ошибка опроса:', err));
12
13    fetch('/api/alarms/active')
14        .then(r => r.json())
```

```
15     .then(data => {  
16         if (data.status === 'ok') {  
17             updateAlarmPanel(data.data);  
18         }  
19     });  
20 }, 1000);
```

Listing 12: Реализация механизма опроса на фронте

Фронтенд параллельно опрашивает конечные точки `/api/sensors/latest` и `/api/alarms/active`, гарантируя синхронизацию числовых карточек, трендовых кривых и списка оповещений с состоянием бэкенда. По сравнению с push-уведомлениями через WebSocket опрос в ущерб реальному времени обеспечивает лёгкость реализации и широкую совместимость с браузерами; в условиях секундного обновления он полностью справляется с задачей. В последующих итерациях возможен переход на SSE или WebSocket для дальнейшего сокращения накладных расходов на соединения.

5.4 Проектирование панели управления

Пользователь через панель управления в браузере непосредственно изменяет параметры виртуальной окружающей среды, наблюдая в реальном времени изменение выходных сигналов датчиков в заданных условиях. Данная возможность особенно важна для демонстраций в аудитории и диагностики неисправностей.

5.4.1 Проектирование механизма межпроцессного взаимодействия

Цикл сбора данных виртуального МК выполняется в независимом потоке Python, а веб-служба Flask работает в главном потоке; для обмена управляющими командами между ними необходим межпоточный канал. Система использует схему IPC на базе совместно используемого файла: `env_control.txt` служит двунаправленным каналом обмена данными.

Выбор файлового IPC обусловлен четырьмя соображениями: во-первых, отсутствие необходимости в дополнительных зависимостях, таких как Redis или ZeroMQ, что снижает сложность системы; во-вторых, содержимое файла сохраняется после перезапуска процесса, обеспечивая естественную персистентность состояния; в-третьих, текстовый формат удобен для прямого просмотра и ручного редактирования разработчиком, что снижает затраты на отладку; в-четвёртых, файловый ввод-вывод является базовой возможностью подавляющего большинства языков программирования,

что минимизирует сопротивление при будущей интеграции гетерогенных модулей.

Файл `env_control.txt` использует простой формат ключ-значение: по одному параметру на строку в формате `ключ=значение`:

```
1 temperature=25.0
2 humidity=60.0
3 pressure=1013.25
4 illuminance=500.0
5 air_quality=100.0
6 noise_level=40.0
7 wind_speed=5.0
```

Listing 13: Пример формата файла управления окружающей средой

В файле `env_control.txt` поддерживаются семь параметров окружающей среды. Поток сбора данных считывает данный файл в начале каждого периода дискретизации и обновляет входные параметры виртуальных датчиков до актуальных значений; веб-бэкенд при получении POST-запроса перезаписывает соответствующие ключи, и новые параметры вступают в силу уже в следующем цикле дискретизации.

5.4.2 Разбор команды `set`

Конечная точка `/api/control` принимает тело POST-запроса в формате JSON, извлекает параметры окружающей среды, подлежащие изменению, и перезаписывает их в `env_control.txt`. Логика разбора следует семантике частичного обновления: обновляются только те ключи, которые присутствуют в теле запроса; остальные ключи сохраняют прежние значения, избавляя пользователя от необходимости каждый раз отправлять полный набор параметров.

```
1 @app.route('/api/control', methods=['POST'])
2 def set_environment():
3     """Устанавливает параметры окружающей среды """
4     data = request.get_json()
5     if not data:
6         return jsonify({'status': 'error', 'message': 'Нет
7             данных'}), 400
8
9     # считывание текущих параметров окружающей среды
10    env = read_env_file()
```

```
11 # обновлениеуказанныхпараметров
12 valid_keys = ['temperature', 'humidity', 'pressure',
13               'illuminance', 'air_quality', 'noise_level',
14               'wind_speed']
15 for key in valid_keys:
16     if key in data:
17         env[key] = float(data[key])
18
19 # записьвфайл
20 write_env_file(env)
21
22 return jsonify({'status': 'ok', 'data': env})
```

Listing 14: Интерфейс установки параметров окружающей среды

Выполнение `set_environment` разбивается на три шага: сначала загружается существующий файл окружающей среды, затем записи перезаписываются в соответствии с парами ключ-значение из запроса, и наконец результат записывается обратно на диск. Белый список `valid_keys` в сочетании с принудительным преобразованием к `float` блокирует недопустимые ключи и ошибки типов.

5.4.3 Ограничение диапазона параметров

Имитация датчиков обладает ограниченной физической устойчивостью к экстремальным входным значениям, поэтому бэкенд для каждого из семи параметров окружающей среды устанавливает допустимый интервал. При отправке значения, выходящего за границы, API возвращает код состояния 400 с сообщением об ошибке, требуя корректировки и повторной отправки. Конкретные верхние и нижние границы и значения по умолчанию приведены в таблице 10.

Таблица 10: Допустимые диапазоны параметров окружающей среды

Имя параметра	Минимум	Максимум	По умолчанию	Единица
temperature	-40.0	+85.0	25.0	°C
humidity	0.0	100.0	50.0	% RH
pressure	300.0	1100.0	1013.25	гПа
illuminance	0.0	65535.0	500.0	люкс
air_quality	0.0	10000.0	100.0	ppm
noise_level	0.0	150.0	40.0	дБ
wind_speed	0.0	100.0	5.0	м/с

Дизайн интервалов для каждого параметра строго соответствует техническим документам реальных датчиков и физическим законам: верхние и нижние границы температуры совпадают с номинальным рабочим диапазоном BMP280; влажность ограничена интервалом 0–100%, поскольку выход за его пределы физически лишён смысла; диапазон давления охватывает экстремальные сценарии от низкого давления в горах до высокого давления на уровне моря; верхняя граница освещённости 65535 люкс точно соответствует полному диапазону BH1750 в режиме H-Res. Эти ограничения перехватывают подавляющее большинство ошибочных вводов, поддерживая обоснованность имитационных выходных данных.

6 Тестирование и верификация системы

Для верификации виртуальной аппаратной платформы STM32, драйверов датчиков, модуля хранения данных, системы оповещений и веб-визуализации необходимо построить систематизированную тестовую среду и выполнить многомерную верификацию. Ниже приведены описания тестовой среды, функционального покрытия, производительностных эталонов, точности датчиков и надёжности оповещений.

6.1 Тестовая среда

Конфигурация тестовой среды системы представлена в таблице [11](#).

Таблица 11: Конфигурация тестовой среды системы

Компонент	Конфигурация
Операционная система	Windows 11 Professional 64-bit
Процессор	Intel Core i7-12700H @ 2.3 ГГц
Память	16 ГБ DDR5
Версия Python	Python 3.11.6
Версия Flask	Flask 3.0.0
Версия SQLite	SQLite 3.42.0
Браузер	Google Chrome 120.0

Тестовый хост — ноутбук на базе Intel Core i7-12700H с 16 ГБ памяти DDR5; данная конфигурация соответствует современным условиям разработки. Весь тестовый цикл выполнялся локально в замкнутом режиме, что исключило влияние задержек глобальной сети и колебаний нагрузки удалённых серверов.

6.2 Функциональное тестирование

Функциональное тестирование поэлементно сверяет поведение системы со спецификацией требований. Дизайн тестовых случаев охватывает шесть модулей: инициализацию виртуального МК, считывание данных с датчиков, загрузку веб-страницы, получение данных через API, срабатывание порогов оповещений и передачу параметров через панель управления. Результаты тестирования сведены в таблицу [12](#).

Таблица 12: Результаты функционального тестирования

№	Тест	Шаги
FT-01	Запуск МК	Инициализация экземпляра VirtualSTM32
FT-02	Включение АЦП	Конфигурация тактирования и бита включения АЦП
FT-03	Считывание DHT11	Вызов метода <code>dht11.read()</code>
FT-04	Считывание MQ-135	Вызов <code>read()</code> после прогрева
FT-05	Считывание BH1750	Отправка команды измерения и считывание
FT-06	Считывание BMP280	Считывание ID чипа и получение данных
FT-07	Считывание шума	Вызов метода <code>noise.read()</code>
FT-08	Доступ к веб-главной	Обращение в браузере к <code>http://localhost:5000/</code>
FT-09	Получение данных API	GET-запрос <code>/api/sensors/latest</code>
FT-10	Срабатывание оповещения о высокой температуре	Установка <code>temperature=40</code>
FT-11	Срабатывание оповещения о низком давлении	Установка <code>pressure=920</code>
FT-12	Операция панели управления	Изменение параметров окружающей среды через веб-интерфейс

Все 12 тестовых случаев, представленных в таблице 12, успешно пройдены. Уровень драйверов датчиков (FT-03 – FT-07) подтвердил корректность возврата физических величин всеми пятью виртуальными датчиками; уровень веб-служб (FT-08, FT-09) подтвердил корректность рендеринга страниц и ответов JSON-интерфейса; уровень логики оповещений (FT-

10, FT-11) подтвердил корректное обнаружение выхода за верхние и нижние пороги; уровень управления (FT-12) подтвердил возможность удалённой перезаписи файла параметров окружающей среды.

6.3 Производительное тестирование

Производительное тестирование сосредоточено на оценке задержек отклика и потребления ресурсов, проверяя, сохраняет ли система способность удовлетворять требованиям мониторинга в реальном времени при длительной непрерывной работе.

6.3.1 Время отклика API

С помощью библиотеки requests Python к различным конечным точкам бэкенда Flask были отправлены 100 последовательных запросов; для каждого запроса фиксировалось время кругового обращения, по которому рассчитывались статистические показатели. Результаты измерений представлены в таблице 13.

Таблица 13: Результаты тестирования времени отклика API

Конечная точка API	Среднее время	Минимум	Максимум
GET /api/sensors/latest	12.5 мс	8.2 мс	28.4 мс
GET /api/sensors/history	45.3 мс	32.1 мс	112.6 мс
GET /api/alarms/active	9.8 мс	6.5 мс	22.1 мс
POST /api/control	15.2 мс	10.3 мс	35.7 мс
GET /api/stats	38.6 мс	25.4 мс	98.3 мс

Среднее время отклика по всем конечным точкам удерживается в пределах 100 мс, что соответствует заданному эталону производительности. Конечные точки /api/sensors/latest и /api/alarms/active выполняют запрос единственной последней записи и требуют в среднем около 10 мс; /api/sensors/history и /api/stats требуют сканирования временного интервала и выполнения агрегатных вычислений, вследствие чего среднее время возрастает до 40–45 мс; дополнительные накладные расходы обусловлены в основном интервальным поиском и группировкой в базе данных.

6.3.2 Стабильность периода сбора данных

Дрожание периода дискретизации непосредственно отражается на плавности трендовых кривых и влияет на временную точность срабатывания оповещений. В ходе эксперимента непрерывно фиксировалось фактическое

время выполнения 100 периодов; в качестве базы использовался целевой период $T_{sample} = 2.0$ с, оценивалась величина дрейфа.

Средний период по 100 дискретизациям составил 2.003 с, стандартное отклонение 0.018 с, максимальный положительный дрейф +0.052 с. Корень дрожания кроется в колебаниях длительности одиночной операции записи в SQLite и логики проверки оповещений; даже при накоплении исторических данных режим WAL позволяет поддерживать время ожидания блокировок на низком уровне. В целом стабильность периода достаточна для поддержки мониторинга в реальном времени; трендовые кривые достоверно воспроизводят изменения параметров окружающей среды.

6.3.3 Использование памяти

С помощью встроенного модуля tracemalloc и сторонней библиотеки psutil Python выполнялось непрерывное отслеживание потребления памяти системой. По истечении 1 часа непрерывной работы детали потребления по модулям представлены в таблице 14.

Таблица 14: Результаты тестирования использования памяти

Модуль	Использование памяти
Веб-служба Flask	около 45 МБ
Виртуальный МК и экземпляры датчиков	около 12 МБ
Поток сбора данных	около 8 МБ
Кэш базы данных SQLite	около 5 МБ
Ресурсы фронтенд-страницы	около 3 МБ (в браузере)
Итого	около 73 МБ

Общее потребление памяти системой составляет около 73 МБ, из которых веб-служба Flask занимает около 45 МБ; данная доля в основном обусловлена средой выполнения интерпретатора Python и внутренними структурами фреймворка Flask. 73 МБ в современных вычислительных условиях являются крайне лёгкой нагрузкой; система способна функционировать без затруднений даже на пограничных устройствах, таких как Raspberry Pi.

6.4 Проверка точности датчиков

Достоверность виртуальной мониторинговой станции в значительной степени определяется точностью имитации датчиков. Для количественной оценки данного показателя выполнено сравнение ошибок виртуальных

датчиков с номинальной точностью, заявленной в официальных технических документах.

6.4.1 Сравнительный анализ точности

Различия в точности между виртуальными и реальными чипами сведены в таблицу 15.

Таблица 15: Сравнение точности виртуальных и реальных датчиков

Датчик	Измеряемый параметр	Реальная точность	Имитационная точность
DHT11	Температура	$\pm 2^{\circ}\text{C}$	$\pm 0.8^{\circ}\text{C}$
DHT11	Влажность	$\pm 5\% \text{ RH}$	$\pm 1.5\% \text{ RH}$
MQ-135	Концентрация газа	$\pm 15\%$	$\pm 8\%$
BH1750	Освещённость	$\pm 20\%$	$\pm 5\%$
BMP280	Давление	$\pm 1 \text{ гПа}$	$\pm 0.5 \text{ гПа}$
BMP280	Температура	$\pm 0.5^{\circ}\text{C}$	$\pm 0.3^{\circ}\text{C}$
Датчик шума	Уровень звукового давления	$\pm 3 \text{ дБ}$	$\pm 1.5 \text{ дБ}$

Как видно из таблицы 15, интервалы ошибок всех виртуальных датчиков уже номинальной точности реальных чипов, указанной в технической документации. Программная имитация исключает физические источники помех, такие как температурный дрейф, старение компонентов, электромагнитная связь и пульсации питания; выходной шум формируется исключительно генератором гауссовских случайных чисел, вследствие чего воспроизводимость и повторяемость значительно превосходят показатели реального оборудования.

Тот факт, что имитационная точность превышает реальную, не снижает учебной и верификационной ценности платформы. В аудиторных условиях низкий уровень шума позволяет студентам легче распознавать закономерности между откликом датчика и переменными окружающей среды, избегая маскировки ключевых особенностей случайными ошибками реального оборудования. При необходимости приближения к реальным условиям достаточно увеличить стандартное отклонение σ гауссовского шума, чтобы вернуть виртуальные датчики к уровню ошибок, близкому к реальным чипам, и проводить тестирование в более практическом ключе.

6.4.2 Долгосрочное тестирование стабильности

Для виртуального BMP280 проведено 24-часовое непрерывное тестирование стабильности: параметры окружающей среды фиксированы на уровне $P = 1013.25 \text{ гПа}$, $T = 25.0^{\circ}\text{C}$, период дискретизации 2 с. Статистика по 43200 точкам дискретизации показала, что среднее значение давления

составило 1013.28 гПа, стандартное отклонение 0.42 гПа, максимальное положительное отклонение +1.8 гПа, максимальное отрицательное отклонение –1.5 гПа. Распределение точек дискретизации подчиняется нормальному закону; монотонного дрейфа или систематического смещения не выявлено, что подтверждает числовую стабильность виртуального датчика при длительной эксплуатации.

6.5 Проверка функции оповещений

Надёжность модуля оповещений определяет, сможет ли пользователь своевременно получить уведомление и предпринять действия при ухудшении окружающей среды. Ниже представлены два типовых сценария для верификации полного цикла срабатывания, отображения и снятия оповещения.

6.5.1 Сценарий оповещения о высокой температуре

Постановка задачи: через панель управления температура окружающей среды повышена до 40°C; данное значение превышает верхний порог температуры 35°C.

Шаги тестирования:

1. Через панель управления установить параметр `temperature` равным 40.0.
2. Дождаться выполнения цикла сбора данных как минимум один раз (около 2 с).
3. Выполнить запрос к таблице `alarm_log` и проверить, создана ли новая запись об оповещении температуры.
4. Открыть панель оповещений веб-интерфейса и подтвердить корректное отображение информации об оповещении.
5. Восстановить `temperature` до 25.0 и дождаться снятия оповещения.

Зарегистрированные наблюдения: после установки `temperature` в 40.0 система во втором цикле дискретизации успешно зафиксировала выход за порог; в таблице `alarm_log` добавлена запись со `status='ACTIVE'` и `alarm_type='HIGH'`; в поле `actual_value` зафиксировано значение 40.1°C (с наложенным имитационным шумом), пороговое значение `threshold` составило 35.0°C. На веб-панели оповещений данная аномалия выделена красной карточкой. После возврата температуры в нормальный интервал примерно через 2 с та же запись автоматически переведена в состояние `RESOLVED`, а в поле `resolved_at` записан момент снятия.

6.5.2 Сценарий оповещения о низком давлении

Постановка задачи: через панель управления атмосферное давление снижено до 920 гПа; данное значение ниже нижнего порога давления 980 гПа.

Шаги тестирования:

1. Через панель управления установить параметр `pressure` равным 920.0.
2. Дождаться выполнения цикла сбора данных как минимум один раз.
3. Выполнить запрос к таблице `alarm_log` и проверить, создана ли новая запись об оповещении давления.
4. Открыть панель оповещений веб-интерфейса и подтвердить корректное отображение информации об оповещении.
5. Восстановить `pressure` до 1013.25 и дождаться снятия оповещения.

Зарегистрированные наблюдения: после установки `pressure` в 920.0 система корректно зафиксировала выход за нижний порог и создала запись об оповещении типа LOW; `actual_value` составило 919.6 гПа. На веб-интерфейсе низкое оповещение отображается оранжевой меткой (высокое оповещение — красной). После восстановления давления оповещение в последующих циклах дискретизации автоматически помечено как RESOLVED.

6.5.3 Сценарий одновременного возникновения нескольких оповещений

Верификация параллельного сценария: одновременное повышение `temperature` до 42.0°C и `air_quality` до 800 ppm, вынуждающее систему сгенерировать HIGH-оповещения по двум независимым измерениям. Система создаёт независимые записи ACTIVE для обоих датчиков, не перезаписывая друг друга на уровне базы данных; на веб-панели оповещений обе аномалии отображаются параллельно, цвета соответствуют датчикам; при последующем снятии каждого из оповещений миграция состояний также происходит независимо. Данный сценарий подтверждает, что модуль оповещений сохраняет логическую корректность и стабильность при многоканальных параллельных условиях.

6.6 Демонстрация веб-интерфейса

Информационная архитектура и визуальная компоновка веб-интерфейса напрямую определяют эффективность получения пользователем информации

о состоянии окружающей среды. Фронтенд системы разделён на шесть функциональных зон:

(1) **Карточки данных датчиков:** на первом экране горизонтально расположены 6 карточек данных; в верхней части каждой карточки указано название датчика, в центре крупным шрифтом отображается текущее значение с единицей измерения, в нижней части присутствует стрелка тренда (↑ рост, ↓ снижение, → стабильность). При нахождении показания в безопасном интервале фон карточки окрашен в зелёный цвет; при приближении к порогу цвет плавно меняется на жёлтый, формируя интуитивную визуальную индикацию.

(2) **График температуры и влажности:** в центральной левой зоне размещён двойной линейный график с двумя осями Y; красная последовательность отслеживает тренд температуры, синяя — тренд влажности, ось X представляет время дискретизации. При наведении курсора на точку данных появляется всплывающая подсказка с точным значением; обновление кривой сопровождается плавной анимацией перехода.

(3) **График качества воздуха и освещённости:** в центральной правой зоне симметрично размещён ещё один двойной линейный график; фиолетовая кривая отображает качество воздуха, жёлтая — освещённость. Учитывая значительную разницу в порядках величин ppm и люкс, независимые шкалы слева и справа предотвращают визуальное сжатие низкоамплитудной последовательности в прямую линию.

(4) **Панель статистической информации:** в нижней левой части сводятся агрегированные показатели для трёх временных масштабов — 1 час, 24 часа и 7 дней, охватывающие средние и экстремальные значения. Данные динамически запрашиваются через /api/stats; пользователь посредством выпадающего меню переключает статистическое окно.

(5) **Панель оповещений:** в нижней правой части размещены две секции: верхняя перечисляет оповещения в состоянии ACTIVE в реальном времени с красным/оранжевым фоном для привлечения внимания; каждое оповещение содержит тип, датчик, момент срабатывания и значение выхода за порог. Нижняя секция отображает исторические записи в обратном хронологическом порядке с поддержкой постраничного просмотра.

(6) **Панель управления:** в правой боковой панели размещены 7 групп регуляторов, соответствующих температуре, влажности, давлению, освещённости, качеству воздуха, шуму и скорости ветра. Рядом с каждым регулятором отображается текущее показание и допустимый интервал; выход за границы блокируется немедленно с выдачей дружелюбного уведомления.

Фронтенд использует адаптивную сеточную компоновку, которая в

зависимости от ширины окна просмотра автоматически перестраивает расположение карточек и графиков, сохраняя читаемость как на десктопе, так и на мобильных устройствах. Цветовая гамма построена на светло-сером фоне с преобладанием синих и зелёных тонов и акцентами красного и оранжевого для оповещений; иерархичность цветов помогает пользователю за 3 секунды локализовать ключевую информацию.

7 Заключение и перспективы

7.1 Итоги работы

Настоящее исследование посвящено виртуализации интеллектуальной станции мониторинга окружающей среды на базе STM32; выполнена полная цепочка работ от теории до прототипа, основные результаты которой можно суммировать в трёх пунктах:

(1) **Построена высокодостоверная виртуальная аппаратная платформа STM32.** В качестве объекта отсчёта использована модель STM32F103C8T6; на языке Python реализован полный набор виртуальной периферии, включающий ядро Cortex-M3, трёхсегментную систему отображения памяти (Flash/SRAM/регистры), таймер SysTick, GPIO, АЦП, I2C, UART и TIM. Данная платформа не зависит от какого-либо физического оборудования и способна выполнять встроенные программы в средах Windows, Linux и macOS. Верификационные эксперименты показали, что поведение виртуальных регистров при чтении и записи высоко согласовано с описанием в справочном руководстве STM32, что вполне достаточно для аудиторных занятий и ранней верификации прототипов.

(2) **Создана унифицированная платформа виртуализации датчиков окружающей среды.** Поверх виртуального оборудования выполнена адаптация драйверов для пяти типовых датчиков окружающей среды: DHT11, MQ-135, BH1750, BMP280 и датчик шума. На уровне платформы унифицированы отображение параметров окружающей среды, пересчёт физических величин и механизм инъекции гауссовского шума $Y = X + N(0, \sigma^2)$, благодаря чему имитационные выходные данные одновременно следуют физическим закономерностям и сохраняют характерные случайные черты реальных устройств. Каждый драйвер полностью воспроизводит протоколы однопроводной шины и I2C, временные характеристики включения и цепи обработки сигналов; комплексная погрешность превосходит номинальную точность, заявленную в официальных технических документах.

(3) Реализована полноценная веб-система визуализации и управления данными. Бэкенд на базе Flask предоставляет RESTful API; SQLite отвечает за персистентное хранение данных датчиков, записей об оповещениях и системных журналов; фронтенд на нативных HTML/CSS/JavaScript в сочетании с Chart.js обеспечивает построение линейных графиков в реальном времени, просмотр истории, всплывающие оповещения и слайдеры управления параметрами окружающей среды. Панель управления и виртуальный МК взаимодействуют через совместно используемый файл `env_control.txt`, реализуя замкнутую связь: пользователь перемещает слайдер на фронтенде, а изменение показаний датчиков наблюдается уже в следующем цикле дискретизации.

Экспериментальная фаза подтвердила: виртуальная платформа способна к круглосуточной стабильной работе в течение 7 дней в неделю; дрейф периода дискретизации контролируется в пределах ± 0.052 с; среднее время отклика веб-API не превышает 100 мс; общее потребление памяти составляет около 73 МБ. Модуль оповещений точно фиксирует выход за верхние и нижние пороги и архивирует события; фронтенд-интерфейс обладает чёткой информационной иерархией. Данное чисто программное решение сводит порог входа для встроенных экспериментов к минимуму и одновременно создаёт среду верификации с быстрой итерацией и нулевым расходом оборудования для проектов Интернета вещей на ранних этапах.

7.2 Перспективы дальнейших исследований

Текущая версия виртуальной мониторинговой станции охватывает ключевые учебные сценарии и верификацию прототипов, однако в инженерной глубине и интеграции с экосистемой остаётся пять направлений, заслуживающих углублённого изучения:

(1) Имитация операционной системы реального времени FreeRTOS. Существующая платформа поддерживает только программы bare-metal; планировщик задач полностью полагается на циклический опрос и прерывания. На следующем этапе возможна адаптация имитации планировщика FreeRTOS с поддержкой вытеснения по приоритетам, кругового роуинга, синхронизации семафорами, обмена сообщениями через очереди и динамического распределения памяти, позволяющая обучающимся осваивать многозадачное параллельное программирование в виртуальной среде и сокращать когнитивный разрыв между аудиторией и промышленной практикой.

(2) Поддержка протокола MQTT. HTTP-опрос при секундном обновлении допустим, но накладные расходы на соединения и задержки push-уведомлений

ограничивают масштабируемость в высоконагруженных сценариях Интернета вещей. После внедрения MQTT виртуальный МК в роли издателя отправляет данные датчиков брокеру, а веб-фронтенд и мобильное приложение в роли подписчиков получают обновления по темам, реализуя истинную доставку по требованию. MQTT как фактический отраслевой стандарт протокола Интернета вещей напрямую повысит способность платформы к интеграции с реальными промышленными системами.

(3) Расширение библиотеки датчиков. Существующая библиотека датчиков охватывает пять измерений: температуру, влажность, давление, освещённость и качество воздуха. В дальнейшем возможно подключение модулей MH-Z19C (CO₂), PMS5003 (частицы), GUVB-S12SD (ультрафиолет), датчиков влажности почвы и дождемеров с качающейся трубкой. После расширения ассортимента платформа сможет бесшовно адаптироваться к вертикальным отраслям, таким как теплицы точного земледелия, городская сетчатая экологическая мониторинговая сеть и охрана труда на производстве.

(4) Имитация режимов низкого энергопотребления. STM32F103C8T6 располагает тремя уровнями энергосбережения: Sleep, Stop и Standby; время автономной работы от батареи в значительной степени зависит от тонкого управления этими состояниями. В дальнейшем возможна имитация в виртуальной платформе полного цикла гейтирования тактирования, отключения периферии и маппинга источников пробуждения, позволяющая обучающимся без расходования реальных батарей понимать компромисс между энергопотреблением и задержкой отклика.

(5) Разработка мобильного приложения. Существующий веб-интерфейс уже обладает кроссплатформенностью, однако в аспектах сенсорного управления на малом экране, фоновой активности и локальных уведомлений остаётся ограниченным песочницей браузера. После разработки сопутствующего приложения на базе Flutter или React Native станет возможным подключение системных push-уведомлений, голосового оповещения об аномалиях и привязка мониторинговых данных к GPS-геопозиции, реализуя по-настоящему мобильное восприятие окружающей среды.

С точки зрения образования виртуализация устраняет жёсткие ограничения закупки оборудования и планирования лабораторных занятий; студент может в любое время на своём компьютере воспроизвести полный цикл разработки встроенных систем. С инженерной точки зрения чисто программная среда верификации позволяет на ранних этапах проекта с минимальными затратами быстро итерировать архитектурные решения. По мере роста отраслевого спроса на специалистов по встроенным системам для Интернета вещей подобные высокодостоверные виртуальные платформы будут играть

всё более ключевую роль в сокращении цикла подготовки кадров и снижении издержек на пробные ошибки.

Список литературы

- [1] Лю Хоулянь, Ян Сэнь. Практическое руководство по разработке библиотек STM32: на базе STM32F4[M]. Пекин: Машиностроительная промышленность, 2019.
- [2] Zheng Dian Yuan Zi (郑典元). Руководство по разработке STM32F1: версия с библиотечными функциями[M]. Гуанчжоу: Zheng Dian Yuan Zi, 2022.
- [3] Ван Лихуа, Чжан Минхуэй. Проектирование и реализация виртуальной экспериментальной платформы для встроенных систем[J]. Компьютерное образование, 2021(5): 112-117.
- [4] Чэнь Чжиган, Ли Сяофэн. Обзор исследований интеллектуальных систем мониторинга окружающей среды на базе Интернета вещей[J]. Датчики и микросистемы, 2020, 39(8): 1-4.
- [5] Чжоу Руныцзин, Чжан Лина. Имитация и применение STM32 на базе Proteus[M]. Пекин: Электронная промышленность, 2018.
- [6] ARM Limited. Справочное руководство по техническим характеристикам Cortex-M3[EB/OL]. ARM Developer, 2021.
- [7] STMicroelectronics. Справочное руководство RM0008: STM32F101х, STM32F102xx, STM32F103xx, STM32F105xx и STM32F107xx продвинутые 32-разрядные МК на базе ARM[EB/OL]. ST Community, 2021.
- [8] Sensirion AG. Техническая документация SHT30-DIS: датчик влажности и температуры[EB/OL]. Sensirion, 2020.
- [9] Bosch Sensortec. Техническая документация цифрового датчика давления BMP280[EB/OL]. Bosch, 2020.
- [10] ROHM Semiconductor. Техническая документация BH1750FVI-E: цифровой 16-разрядный датчик окружающего освещения с последовательным выходом[EB/OL]. ROHM, 2019.
- [11] Hanwei Electronics. Техническая документация газового датчика MQ-135[EB/OL]. Hanwei, 2018.
- [12] Flask Documentation. Flask 3.0 Quick Start[EB/OL]. Pallets Projects, 2023.
[urlhttps://flask.palletsprojects.com/](https://flask.palletsprojects.com/)

- [13] Mozilla Developer Network. Fetch API Web APIs[EB/OL]. MDN Web Docs, 2023.
urlhttps://developer.mozilla.org/en-US/docs/Web/API/Fetch_API
- [14] Chart.js Documentation. Chart.js 4.x Getting Started[EB/OL]. Chart.js, 2023.
url<https://www.chartjs.org/docs/>
- [15] Ван Чжихуа, Лю Ян. Проектирование и реализация системы мониторинга окружающей среды Интернета вещей на базе протокола MQTT[J]. Технологии Интернета вещей, 2022, 12(3): 45-48.
- [16] Ли Цзяньго, Чжан Вэй. Исследование применения технологий имитации в обучении встроенным системам[J]. Журнал электротехнического и электронного образования, 2021, 43(2): 78-82.
- [17] SQLite Consortium. Документация SQLite[EB/OL]. SQLite, 2023.
url<https://www.sqlite.org/docs.html>
- [18] Чжао Минхуа, Сунь Ли. Применение технологий виртуальных приборов в преподавании датчиков[J]. Экспериментальная техника и управление, 2020, 37(6): 156-160.
- [19] QEMU Project. Документация QEMU[EB/OL]. QEMU, 2023.
url<https://www.qemu.org/documentation/>
- [20] Ян Лицян, Ван Хайтао. Проектирование удалённой веб-платформы для экспериментов по встроенным системам[J]. Компьютерная инженерия и приложения, 2019, 55(14): 245-250.